

Restify

<https://github.com/jCanay/restify>

Nombre Alumno/a: Jaime Canay Agho

Curso: 2º DAM

Módulo: Proyecto Final Ciclo

Contenido

1. Introducción	3
2. Objetivos	3
3. Situación previa.....	4
4. Tecnologías utilizadas	4
5. Solución propuesta.....	6
6. Planificación del proyecto.....	6
6.1 Planificación con diagrama de Gantt	6
6.2 Modelo de desarrollo.....	7
6.3 Análisis del proyecto.....	7
6.3.1 Diagrama de casos de uso	7
6.3.2 Universo de discurso de la base de datos	8
6.3.3 Diagrama Entidad-Relación.....	10
6.4 Diseño del proyecto.....	10
6.4.1 Modelo relacional de la base de datos	10
6.4.2 Diagrama de clases.....	11
6.4.3 Diagramas de flujo de cada caso de uso	12
6.4.4 Mockups de la interfaz	36
6.1 Presupuesto completo (Hardware, software y recursos humanos).....	40
7. Desarrollo y ejecución	43
7.1 Descripción General del Funcionamiento.....	44
7.2 Dificultades y resoluciones técnicas	44
8. Conclusiones y reflexiones.....	46
8.1 Valoración global del proyecto.....	46
8.2 Resumen de lo aprendido.....	47
8.3 Futuras mejoras y ampliaciones	47
9. Bibliografía e Webgrafía.....	48
10. Anexo I – Manual técnico de instalación o puesta en marcha	51
11. Anexo II – Documentación de uso (manuales de usuario)	51

1. Introducción

Restify es una solución tecnológica integral diseñada específicamente para el sector de la restauración que unifica las capacidades de un sistema de planificación de recursos empresariales (**ERP comercial**) con una plataforma interactiva orientada al cliente final para la gestión autónoma de **reservas y pedidos en línea**. El proyecto surge ante la imperante necesidad de digitalizar y cohesionar los flujos operativos de la hostelería moderna bajo un único ecosistema digital centralizado.

A diferencia de los sistemas tradicionales, Restify ha sido concebido desde sus cimientos como una arquitectura desacoplada y modular. Por un lado, dota al establecimiento de un panel de administración dinámico y personalizable enfocado en el control de accesos basado en roles (RBAC), la gestión analítica del local, y la monitorización de comandas en tiempo real. Por otro lado, ofrece una interfaz pública e intuitiva donde los usuarios y clientes habituales pueden consultar la oferta gastronómica, interactuar con el carrito de la compra y gestionar sus reservas de forma directa, ágil y transparente. Esta confluencia 360° optimiza los tiempos de respuesta del personal, maximiza la rotación de las mesas y reduce a cero las fricciones informativas entre la sala, la cocina y el cliente.

2. Objetivos

El diseño y desarrollo de **Restify** se rige por un conjunto de objetivos estratégicos orientados a mitigar los retos operativos que sufre la hostelería actual. En esta etapa, se delimitan las metas funcionales orientadas tanto a la experiencia del cliente externo como a la gestión interna del restaurante, sentando las bases de lo que constituirá el producto final.

A. Experiencia del Cliente (Experiencia en Línea)

- **Motor de Reservas Inteligente:** Implementar la lógica de asignación automática de mesas basada en la disponibilidad real del inventario físico, permitiendo confirmaciones instantáneas sin intervención humana.
- **Facilidad para pedir en línea:** Se planificará una interfaz pública donde los usuarios puedan explorar la carta de productos digitalizados, añadir platos de forma interactiva a un carrito de la compra, actualizar cantidades y proceder a un proceso de confirmación (*checkout*) ágil y sin fricciones.
- **Acceso Seguro y Personalizado:** Se proyectará un sistema de registro e inicio de sesión para que los clientes dispongan de su propia cuenta privada. Esto les permitirá guardar de forma segura sus datos frecuentes (como direcciones de entrega múltiples y datos de contacto) para agilizar futuros pedidos y consultas de su historial.
- **Pasarela de Pagos:** Integrar un sistema de pagos seguro (como Stripe o PayPal) para permitir el cobro de pedidos online y la gestión de fianzas en reservas de grupos grandes.
- **Gestión de Perfil de Cliente:** Desarrollar un área personal donde el usuario pueda consultar su histórico de pedidos, gestionar alérgenos y guardar sus métodos de pago preferidos.

B. Gestión Interna (ERP Avanzado)

- **Análíticas Visuales (BI):** Desarrollar widgets avanzados que generen gráficas comparativas de ventas, horas punta y platos más rentables directamente en el Dashboard.
- **Notificaciones en Tiempo Real:** Implementar WebSockets para que el personal de cocina reciba avisos instantáneos cada vez que un cliente realiza un pedido desde la web.

- **Flexibilizar el Control Horario y Restricciones del local:** Se planteará una sección específica para que los encargados puedan modificar de manera dinámica los horarios de apertura, bloquear días festivos, aplicar cierres por eventos privados o ajustar las horas punta de cocina, haciendo que el portal del cliente se adapte instantáneamente a la realidad operativa del local.

C. Infraestructura y Seguridad

- **Despliegue en la Nube (Cloud):** Migrar la arquitectura de Docker Compose local a un entorno de producción real (como AWS o Azure) utilizando orquestadores de contenedores.
- **Sistema de Auditoría:** Implementar un log de actividades que registre qué administrador o empleado realizó cambios críticos en la configuración del restaurante o en los precios del menú.
- **Optimización de Carga:** Configurar estrategias de caché (como Redis) para acelerar la respuesta de la API ante un volumen alto de peticiones simultáneas de clientes.

3. Situación previa

La motivación de **Restify** surge al observar que muchos restaurantes operan con procesos fragmentados: utilizan una aplicación para reservas, otra para el TPV físico y, a menudo, procesos manuales para pedidos a domicilio. Esta falta de integración provoca pérdida de datos e ineficiencia operativa.

Problema a resolver: La desconexión entre la demanda externa (clientes online) y la gestión interna (inventario y disponibilidad de mesas).

Investigación de alternativas:

- Plataformas de terceros (Marketplaces): Ofrecen visibilidad, pero cobran altas comisiones y no gestionan la logística interna del local.
- Plataformas de Reserva Especializadas: Centralizan bien las reservas de mesas, pero cobran un canon fijo por comensal sentado y actúan como intermediarios.
- Software ERP tradicional: Suelen ser complejos, costosos y carecen de una interfaz amigable para que el cliente final interactúe directamente sin intermediarios.

4. Tecnologías utilizadas

Para la construcción de **Restify**, se ha seleccionado un conjunto de tecnologías de vanguardia que garantizan un rendimiento óptimo, una interfaz de usuario pulida y una arquitectura escalable.

Backend (Arquitectura Robusta)

- **Spring Boot 4:** Framework base para la creación de la API REST de alto rendimiento.
- **JWT (JSON Web Tokens):** Implementación de seguridad *stateless* para la autenticación y autorización segura de usuarios y administradores.
- **Lombok:** Utilizado para reducir el código repetitivo (*boilerplate*), mejorando la legibilidad y el mantenimiento de las entidades y DTOs.
- **Model Mapper:** Librería clave para la conversión eficiente entre objetos de base de datos (Entidades) y objetos de respuesta (DTOs), asegurando una separación clara de capas.

Frontend (Interfaz de Usuario Avanzada)

- **React 18:** Biblioteca principal para la construcción de una interfaz de usuario reactiva y basada en componentes.
- **Tailwind CSS & Shadcn/ui:** Combinación de diseño para lograr una estética profesional, consistente y altamente personalizable mediante componentes de alta calidad.
- **React Grid Layout & React Resizable Panels:** Tecnologías fundamentales que permiten al administrador arrastrar, soltar y redimensionar los elementos del Dashboard, ofreciendo una experiencia de usuario tipo "escritorio".
- **Lucide React:** Set de iconos vectoriales para una navegación visual clara e intuitiva.
- **Axios:** Cliente HTTP encargado de la comunicación con el backend, configurado con interceptores para gestionar el refresco automático de tokens.
- **React Router:** Gestión de la navegación SPA (*Single Page Application*) permitiendo rutas diferenciadas para clientes y administración.
- **Overlay Scrollbars:** Implementación de barras de desplazamiento personalizadas y estéticas que mantienen la coherencia visual en todos los navegadores.

Infraestructura y Persistencia

- **MySQL:** Sistema de gestión de base de datos relacional para el almacenamiento persistente de pedidos, reservas y usuarios.
- **Docker & Docker Compose:** Herramientas de containerización que aseguran que todo el ecosistema (DB + API) se despliegue de forma idéntica en cualquier entorno.

Desafío Común	Limitación en ERPs Antiguos	Solución con Restify (Stack Moderno)
Rigidez de Interfaz	Menús estáticos y difíciles de navegar.	React Grid Layout + Resizable Panels: Dashboard totalmente personalizable por el usuario.
Seguridad de Sesión	Sesiones por cookies propensas a ataques o cierres inesperados.	JWT + Refresh Token: Autenticación persistente, segura y con rotación automática de sesión.
Transferencia de Datos	Acoplamiento entre Base de Datos e Interfaz (poco seguro).	Model Mapper (DTOs): Filtrado estricto de datos; solo se envía al cliente lo estrictamente necesario.
Estética y Usabilidad	Diseño industrial tosco y poco intuitivo.	Shadcn + Tailwind + Lucide: Interfaz minimalista y profesional, similar a las herramientas SaaS actuales.
Comunicación Cliente-Local	El cliente debe llamar por teléfono para reservar o pedir.	Portal de Reservas Online: Integración directa de la demanda externa en el flujo de trabajo del personal.
Instalación Compleja	Requiere configuración manual de bases de datos y servidores locales.	Docker & Docker Compose: Despliegue inmediato y consistente con un solo comando en cualquier máquina.

5. Solución propuesta

La solución de **Restify** se diferencia por su enfoque en la **soberanía de datos** y la **personalización**:

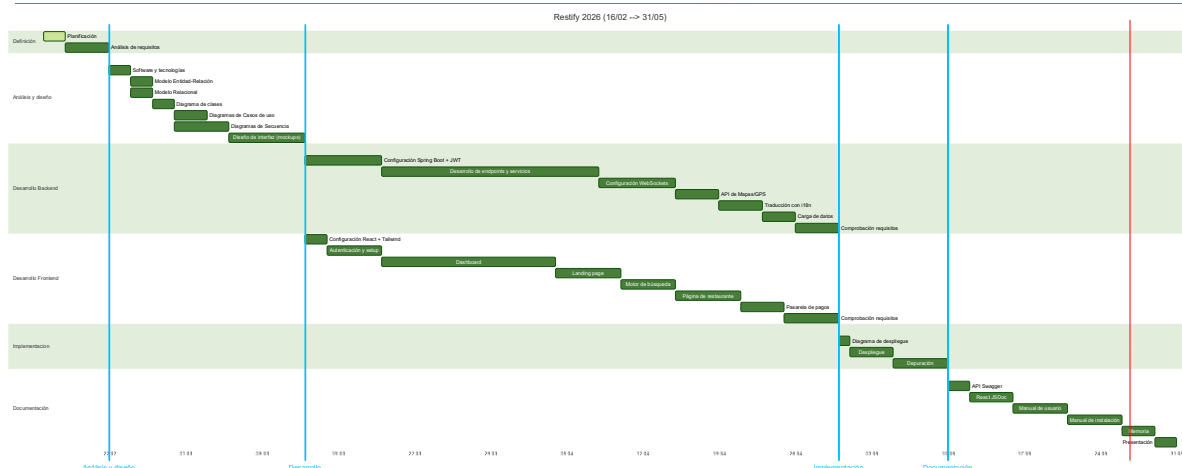
- **Centralización:** A diferencia de otras alternativas, Restify une el portal de ventas (cliente) con la herramienta de gestión (dueño) en una sola base de datos.
- **Interfaz Modular:** El uso de un Dashboard basado en widgets permite que el administrador configure su espacio de trabajo según sus prioridades del día, algo poco común en ERPs rígidos.
- **Seguridad de Grado Empresarial:** La implementación de la rotación de Refresh Tokens asegura que, aunque el sistema sea accesible para clientes externos, la integridad de la gestión interna permanezca protegida.
- **Sin Intermediarios:** Al ser una solución propia, el restaurante elimina las comisiones por pedido y mantiene una relación directa con su base de clientes.

6. Planificación del proyecto

- Planificación do proxecto cun diagrama de Gantt
- Descripción do modelo de desenvolvemento de software a implementar.
- Análise do proxecto:
 - Diagrama de casos de usos.
 - Universo de discurso da base de datos.
 - Diagrama de Entidad-Relación.
- Deseño do proxecto:
 - Modelo relacional da base de datos.
 - Diagrama de clases.
 - Diagramas de fluxo de cada caso de uso.
 - Mockups da interface.
- Presupuesto completo (Hardware software e recursos humans)

6.1 Planificación con diagrama de Gantt

Para garantizar la viabilidad del proyecto y un control riguroso de los plazos de entrega, se ha llevado a cabo una planificación temporal detallada de todas las fases del desarrollo de Restify. El ciclo de vida del proyecto se ha estructurado en bloques secuenciales y solapados que abarcan desde la investigación inicial, pasando por el diseño, hasta la implementación del código. Cada fase cuenta con hitos específicos y una asignación de tiempos estimada para mitigar riesgos e imprevistos.



Esta planificación permite mantener un flujo de trabajo continuo, minimizando los cuellos de botella mediante el desarrollo paralelo de la API REST y la interfaz de usuario. Este calendario sirve como hoja de ruta fundamental para evaluar el progreso real del proyecto frente a las estimaciones iniciales.

6.2 Modelo de desarrollo

Para el desarrollo de Restify se ha optado por un modelo de **desarrollo ágil** basado en una adaptación de Scrum/Kanban, combinado con un enfoque de desarrollo iterativo e incremental.

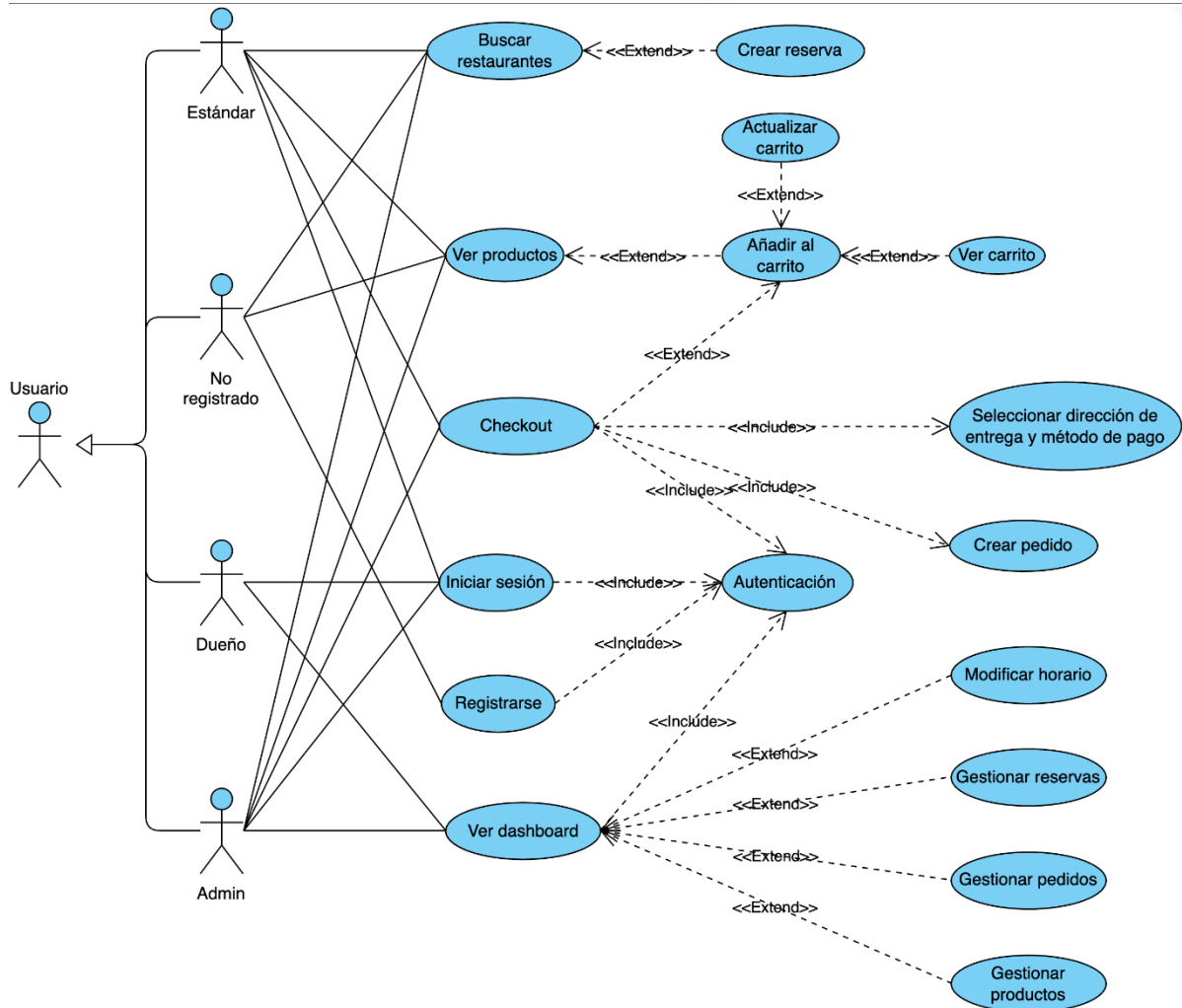
La naturaleza de una aplicación de gestión hostelera (ERP y Dashboard) requiere la validación constante de sus funcionalidades operativas (como la gestión de comandas o reservas). Utilizar un enfoque incremental permite construir un **"Producto Mínimo Viable" (MVP)** completamente funcional —la estructura del backend en Spring Boot y la autenticación segura— para, posteriormente, añadir módulos complejos como las métricas analíticas o la sincronización en tiempo real de la sala.

Las tareas se gestionan mediante un tablero visual donde se prioriza el ciclo de vida del software en base a **Historias de Usuario**, garantizando que cada incremento de código sea estable, testeable y esté listo para ser contenerizado.

6.3 Análisis del proyecto

6.3.1 Diagrama de casos de uso

El análisis funcional de Restify parte de la identificación de los distintos actores que interactúan con el sistema y las fronteras de la aplicación. El **diagrama de casos de uso** define el comportamiento esperado del software desde el punto de vista del usuario, detallando los **permisos**, las restricciones del **Control de Acceso Basado en Roles (RBAC)** y los **flujos principales de trabajo** del personal del restaurante (administradores, camareros, cocina, etc.).



Este mapeo de casos de uso asegura que el sistema cubra todas las necesidades operativas del establecimiento, delimitando de forma clara qué acciones requieren autenticación previa mediante JWT y qué dependencias o extensiones existen entre las diferentes funcionalidades.

6.3.2 Universo de discurso de la base de datos

El sistema está diseñado como una plataforma multi-inquilino (*multi-tenant*) orientada a la gestión de cadenas o grupos de restauración, donde un mismo cliente corporativo puede administrar uno o varios establecimientos, controlando además paneles visuales analíticos configurables por el usuario.

A continuación, se describen las reglas de negocio, entidades y restricciones que componen el universo de discurso modelado en el sistema:

1. Gestión de Usuarios, Cuentas y Seguridad

- Usuarios y Roles:** El acceso al sistema está protegido por credenciales. Un **usuario** (users) se define mediante un nombre de usuario único, correo electrónico, contraseña encriptada y un estado de habilitación (*enabled*). Cada usuario tiene asignado obligatoriamente un único **rol de usuario** (user_roles) (por ejemplo, Administrador, Gerente, Camarero), el cual determina sus permisos globales en la plataforma.

- **Perfiles o Cuentas:** Cada usuario tiene asociado un perfil o **cuenta** (accounts) que almacena su información personal: nombre, apellidos y la ruta de su avatar de usuario. Una cuenta pertenece exclusivamente a un usuario.
- **Direcciones:** Las cuentas de los usuarios y los restaurantes necesitan almacenar ubicaciones físicas. Una **dirección** (addresses) contiene campos específicos como calle, piso, ciudad, país, código postal, zona, notas de entrega, coordenadas geográficas (latitud y longitud) y un indicador de si es la dirección por defecto. Una dirección puede estar vinculada a una cuenta o a un restaurante específico.

2. Estructura del Restaurante y Operaciones

- **Restaurantes:** La entidad central del negocio es el **restaurante** (restaurants). Cada establecimiento cuenta con un nombre, un radio de entrega (*delivery radius*), un indicador de si es el local predeterminado, y está ligado operativamente a una cuenta administradora y a un panel de control o *dashboard*.
- **Productos:** Cada restaurante ofrece una carta de **productos** (products). De cada producto se registra su nombre, descripción, precio, categoría (bebida, entrante, principal, etc.) y el restaurante al que pertenece.
- **Comandas / Pedidos:** Los clientes realizan **pedidos** (orders) en un restaurante. Una comanda registra su estado actual (mediante un tipo numerado o *enum* como 'PENDIENTE', 'PREPARANDO', 'ENTREGADO'). Una comanda vincula de forma transaccional a un restaurante, a la cuenta que realiza o gestiona el pedido, y al producto solicitado (gestionando las líneas de detalle de los artículos consumidos).
- **Reservas:** Los clientes pueden agendar mesas mediante **reservas** (bookings). Una reserva almacena la fecha y hora programada, la fecha de creación del registro y su estado actual (*enum*). Cada reserva se asocia a un restaurante concreto y a la cuenta del cliente que la solicita.

3. Estructura del Restaurante y Operaciones

- **Reglas de Disponibilidad:** Para gestionar los horarios de apertura, cierres por festivos o turnos especiales de cada restaurante, se definen **reglas de disponibilidad** (availability_rules). Estas reglas determinan el día de la semana, hora de apertura y cierre, rangos de fechas de validez (fecha de inicio, fin, efectividad y expiración) y un flag para indicar si el local permanece cerrado (*is_closed*).
- **Tipos de Reglas:** Cada regla de disponibilidad pertenece a un **tipo de regla** (rule_types), el cual define su prioridad (mediante un valor entero para resolver conflictos entre horarios), un nombre y una descripción.

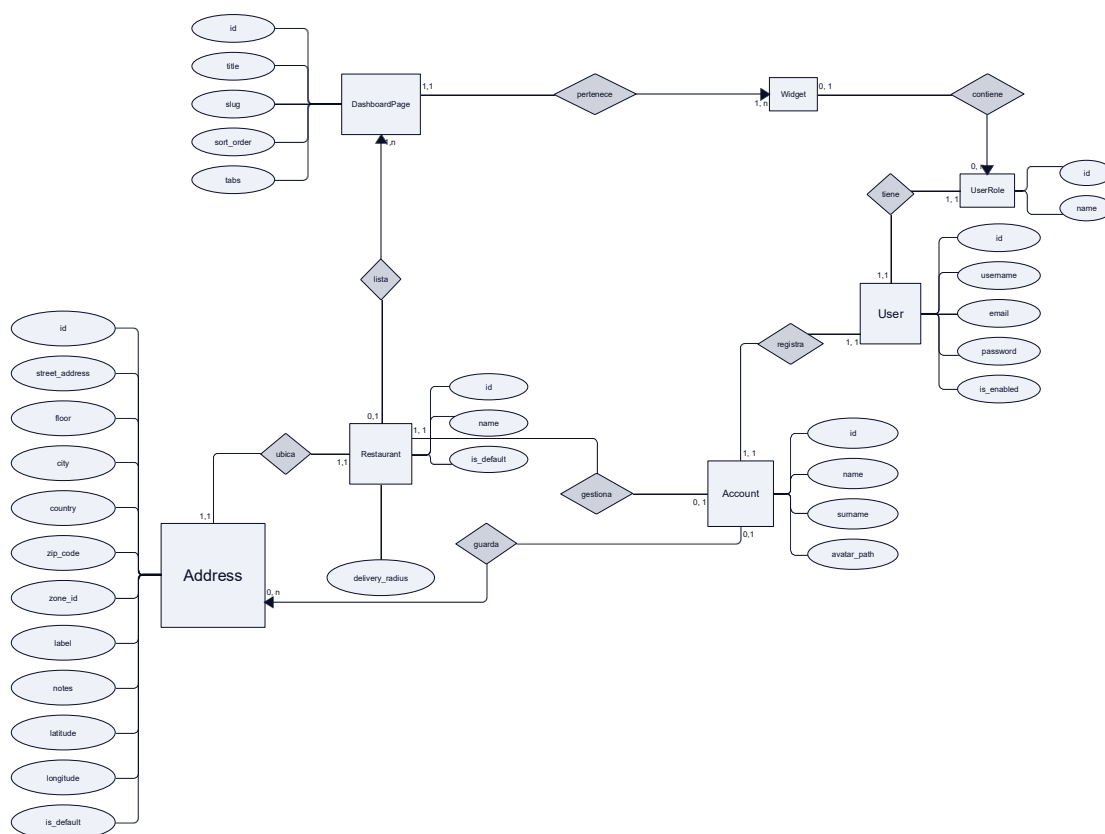
4. Arquitectura del Panel de Control (Dashboards y Widgets)

- **Paneles de Control:** Cada restaurante tiene asociado un **dashboard** (dashboards) principal para la visualización de métricas analíticas del negocio.
- **Páginas del Panel:** Un *dashboard* puede organizarse en múltiples **páginas** (dashboard_pages). Cada página cuenta con un título, un *slug* para la navegación web, un orden de clasificación, una configuración de pestañas (*tabs* en formato JSON) y la referencia al *dashboard* padre.
- **Componentes Visuales (Widgets):** Las páginas contienen **widgets** (widgets) dinámicos. Un widget almacena su tipo (*enum* para gráficos, tablas, contadores), una estructura de diseño y posicionamiento (*layouts* en formato JSON) y la página a la que pertenece.

- **Control de Acceso a Widgets:** La visualización de las métricas es sensible. Por ello, se implementa una tabla de ruptura (`widget_access_roles`) que define qué **roles de usuario** tienen permisos de acceso a qué **widgets** específicos, garantizando que un camarero y un gerente vean paneles adaptados a sus responsabilidades.

6.3.3 Diagrama Entidad-Relación

A partir del universo de discurso y las reglas de negocio analizadas, se ha diseñado el modelo conceptual de datos. El **Diagrama de Entidad-Relación (ERD)** plasma de forma abstracta las entidades principales del sistema, sus atributos y las cardinalidades que rigen sus conexiones, garantizando la integridad de la información desde la fase de diseño.

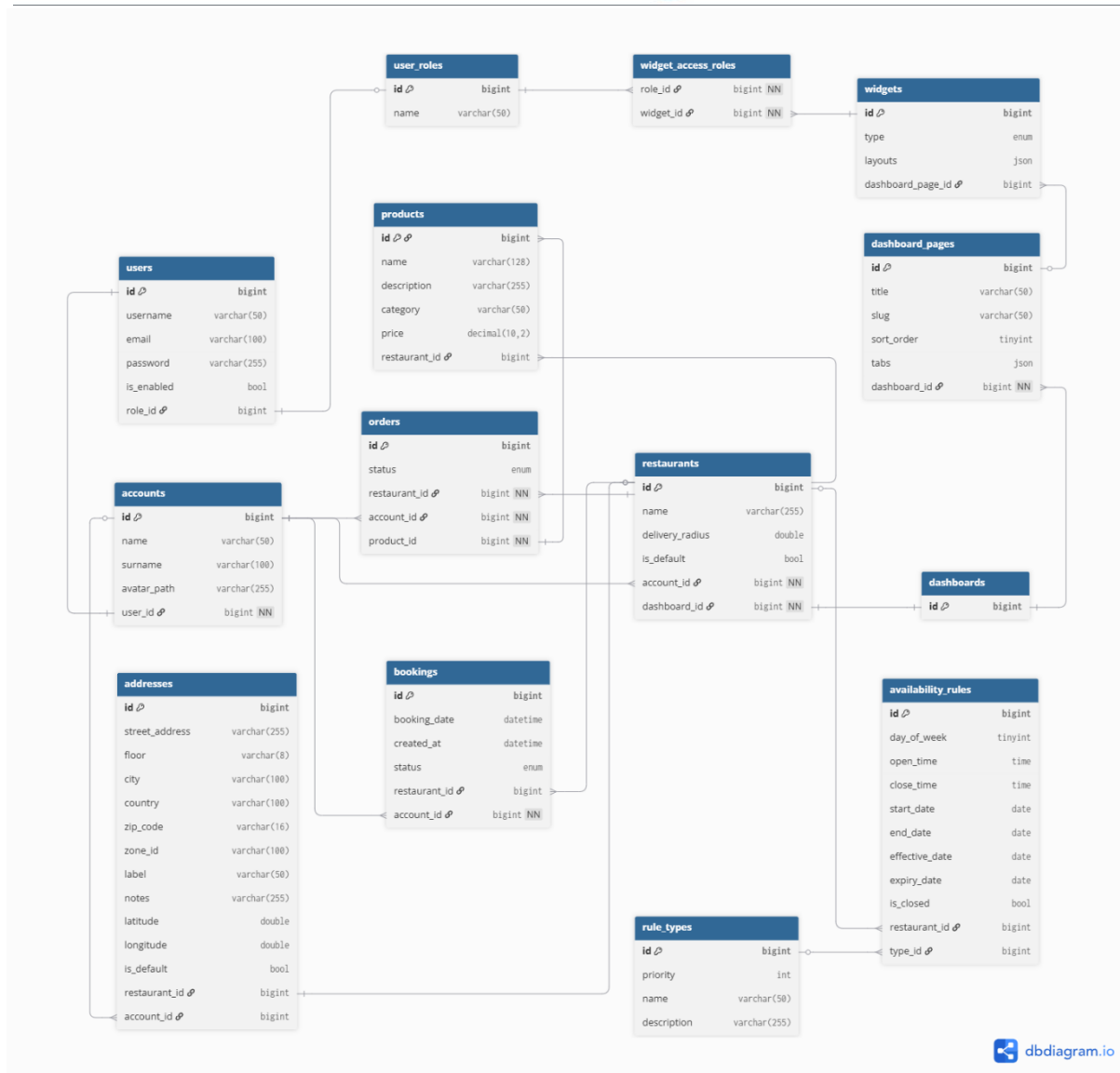


El esquema conceptual presentado sienta las bases para una persistencia de datos robusta, reflejando fielmente cómo interactúan las entidades maestras (como Usuarios y Productos) con las entidades transaccionales (como Comandas y Reservas).

6.4 Diseño del proyecto

6.4.1 Modelo relacional de la base de datos

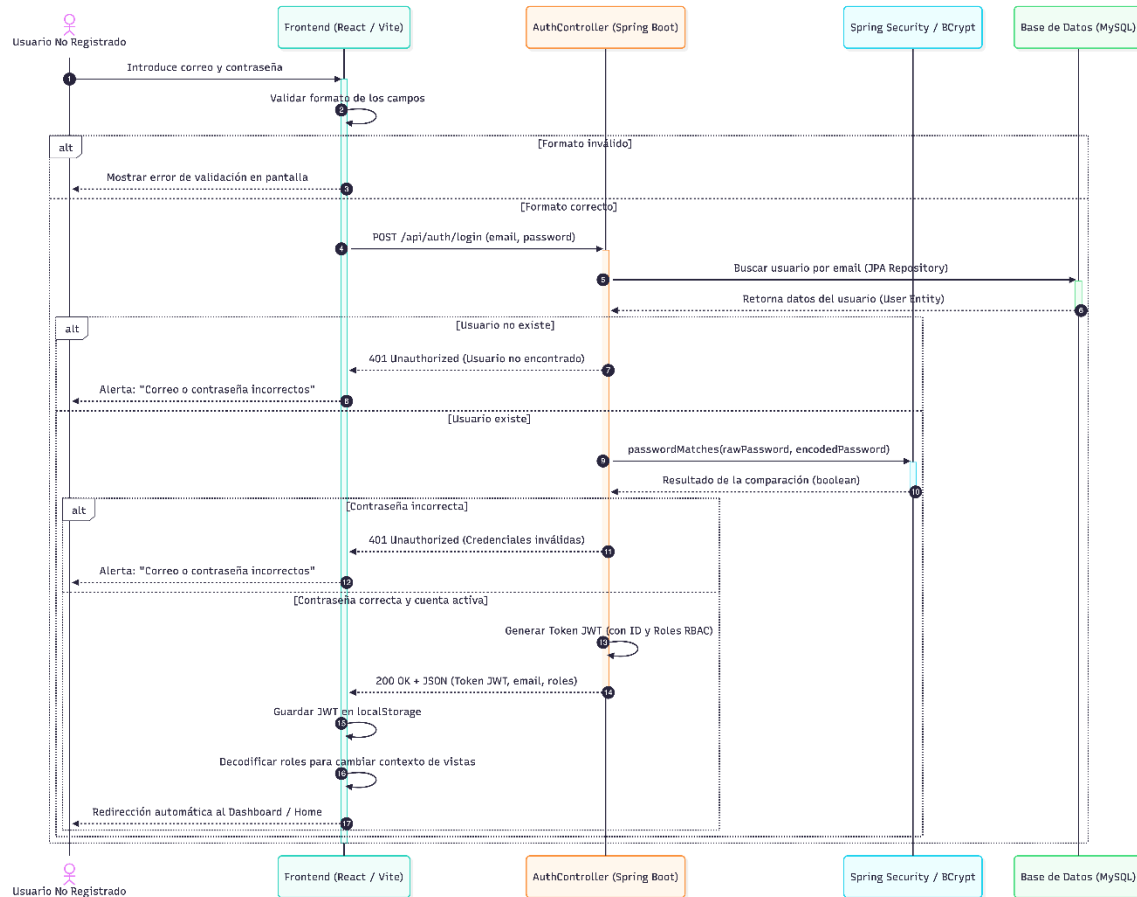
Una vez validado el modelo conceptual, se realiza la transformación al modelo lógico relacional. En esta fase se normalizan las tablas (aplicando las **Formas Normales** para evitar redundancias y anomalías de modificación), se definen formalmente las claves primarias (PK), las claves ajenas o foráneas (FK) y las restricciones de integridad referencial que posteriormente se implementarán en **MySQL**.



Este diseño relacional optimiza las consultas y asegura que las operaciones en cascada (como la eliminación o actualización de registros vinculados) se ejecuten de manera segura, sirviendo como mapa directo para el mapeo objeto-relacional (ORM) mediante Spring Data JPA.

6.4.2 Diagrama de clases

El diseño de la arquitectura orientada a objetos de **Restify** se detalla en el siguiente diagrama de clases. En él se representan los componentes del backend estructurados bajo el patrón arquitectónico multicapa (Controladores, Servicios, Repositorios y Entidades). Se detallan las propiedades, los métodos principales, los tipos de datos y los modificadores de acceso.

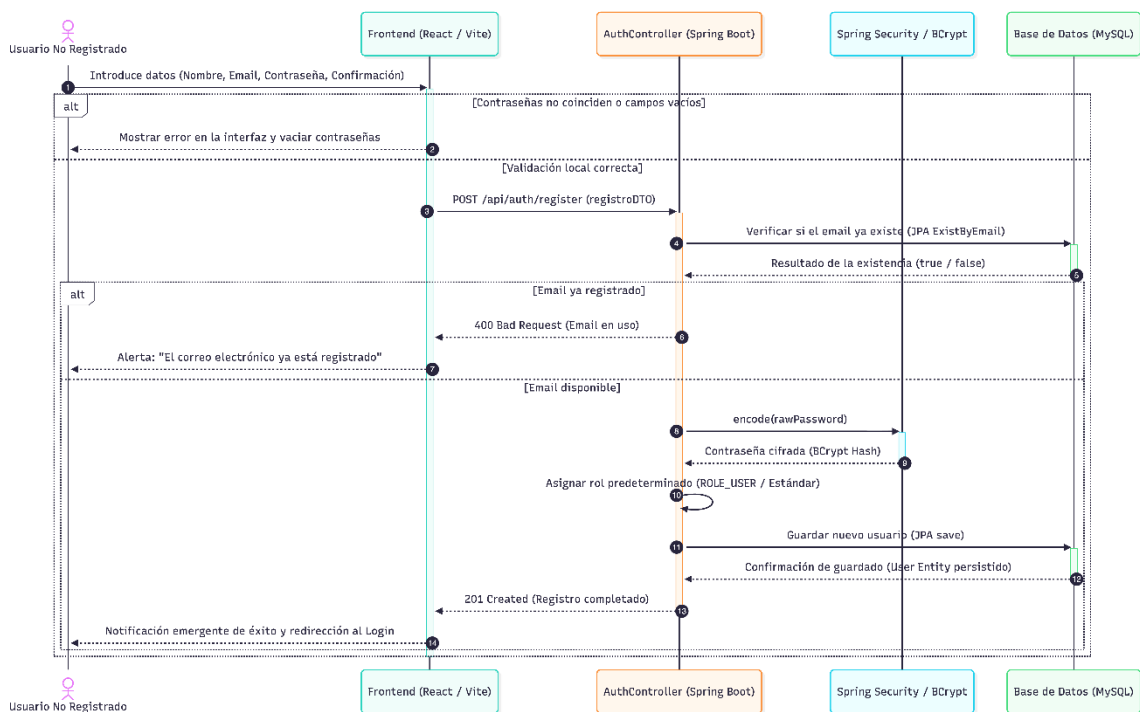


CU-01	Iniciar sesión en la plataforma		
Descripción	Un usuario introduce sus credenciales con el fin de identificarse en la aplicación. Tras una validación correcta, obtendrá un token de acceso que le permitirá acceder a las funcionalidades correspondientes a su rol (Estándar, Dueño o Admin).		
Actores	Admin, Dueño, Estándar.		
Pre condiciones	El usuario debe estar previamente registrado en la base de datos del sistema.		
Post condiciones	El usuario queda autenticado en el sistema. El frontend almacena el token JWT de forma persistente y el sistema le redirige a la pantalla inicial o al dashboard correspondiente según su rol asignado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario selecciona la opción de "Iniciar sesión" en la interfaz de la aplicación web.	El sistema muestra el formulario de inicio de sesión solicitando las credenciales (correo electrónico y contraseña).

	2	El usuario introduce su correo electrónico y su contraseña, y hace clic en el botón de "Enviar".	El sistema recibe los datos, encripta la contraseña introducida y la compara con el hash almacenado en la base de datos MySQL.
		2.1. Si el usuario introduce un correo electrónico con formato inválido el sistema deberá mostrar un mensaje de error de validación en tiempo real debajo del campo del formulario ("Formato de correo no válido") y bloquear el botón de envío hasta que se corrija.	
	3		El sistema valida que las credenciales son correctas, genera un token JWT firmado que codifica el ID del usuario y sus roles asignados (RBAC), y lo envía de vuelta al cliente.
	4	El frontend del navegador recibe el token JWT, lo almacena localmente en la sesión (localStorage/Cookies) para mantener la sesión activa y decodifica los roles del payload.	El sistema destruye el estado de "No registrado" de la sesión actual y redirige al usuario automáticamente a su pantalla de inicio correspondiente (la vista de productos si es cliente Estándar o el panel de control si es Dueño o Admin).
		4.1 Si el usuario autenticado posee el rol de 'Dueño' o 'Admin' el sistema deberá redirigirlo directamente al caso de uso Ver dashboard en lugar de a la vista general de productos de la aplicación.	
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que las credenciales introducidas no coincidan con ningún registro activo el sistema deberá denegar el acceso, limpiar el campo de la contraseña y mostrar un mensaje de alerta genérico: "Correo electrónico o contraseña incorrectos".	En el caso de que la cuenta del usuario se encuentre bloqueada o suspendida el sistema deberá denegar el acceso y mostrar el mensaje: "Esta cuenta ha sido deshabilitada. Contacte con el administrador".
	q	En el caso de que [el servidor backend no responda o la base de datos esté caída] el sistema deberá capturar el error de red en el cliente (Axios) y mostrar una notificación emergente (Toast) indicando: "Error de conexión con el servidor. Inténtelo más tarde".	

Rendimiento	El sistema deberá realizar las acciones descritas en los pasos 2 al 3 (procesamiento en Spring Security, generación del JWT y respuesta de la API) en un máximo de 1.5 segundos bajo condiciones normales de red.
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 3 a 5 veces al día por usuario único, concentrándose la mayor carga en las horas pico de comidas y cenas.
Importancia	Vital (Si este caso de uso falla, el resto de la aplicación privada queda completamente inaccesible debido a las restricciones de Spring Security).
Urgencia	Inmediatamente (Es el pilar fundamental sobre el que se apoya el control de accesos por tokens).
Comentarios	Las contraseñas deben viajar siempre cifradas y en la base de datos se almacenarán utilizando el algoritmo de hash BCrypt provisto por Spring Security. El token generado tendrá un tiempo de expiración limitado determinado en el archivo application.properties.

2. Registrarse

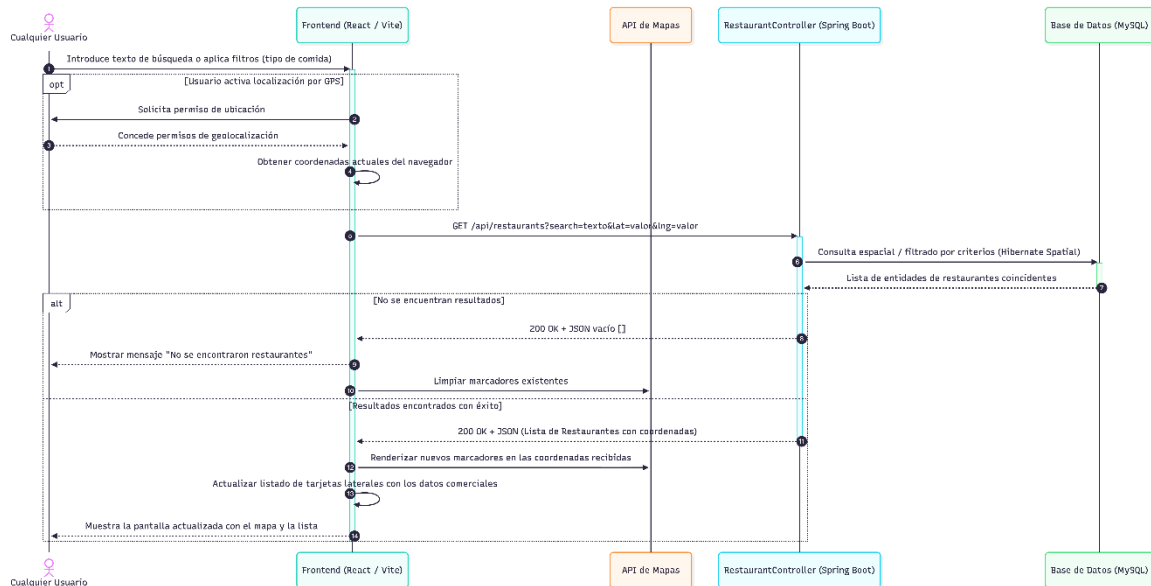


CU-02	Registrar un nuevo usuario en la plataforma
Descripción	Un usuario no registrado introduce sus datos personales y credenciales para darse de alta en el sistema. Al completarse con éxito, se crea su cuenta en la base de datos con el rol predeterminado de cliente para que pueda operar en la aplicación.

Actores	Usuario no registrado		
Pre condiciones	El usuario no debe estar autenticado en la plataforma y debe disponer de una dirección de correo electrónico válida que no haya sido utilizada previamente en el sistema.		
Post condiciones	Se almacena un nuevo registro en la tabla de usuarios de la base de datos MySQL con la contraseña encriptada. El usuario es redirigido automáticamente a la pantalla de inicio de sesión con su cuenta ya activa.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario selecciona la opción de registrarse en la barra de navegación o en la pantalla de bienvenida.	El sistema renderiza el formulario de registro solicitando nombre completo, correo electrónico, contraseña y confirmación de la contraseña.
	2	El usuario rellena los campos solicitados con sus datos y hace clic en el botón de registrarse.	El sistema recibe los datos mediante una petición HTTP POST en el backend y valida que el formato del correo sea correcto, que la contraseña cumpla los requisitos mínimos de seguridad y que ambas contraseñas coincidan.
		2.1. Si el usuario introduce contraseñas que no coinciden, el sistema deberá bloquear el envío del formulario, vaciar los campos de contraseña y mostrar un mensaje de error advirtiendo al usuario en la interfaz.	2.2. Si la contraseña introducida no cumple con los criterios de complejidad mínimos, el sistema deberá mostrar una alerta indicando los requisitos faltantes (como longitud mínima o caracteres especiales).
	3		El sistema realiza una consulta en la base de datos para comprobar que el correo electrónico introducido no pertenece a ninguna cuenta existente.
	4		El sistema genera el hash de la contraseña utilizando BCrypt, asigna el rol base de usuario estándar y guarda el nuevo registro de manera persistente en MySQL.
	5		El sistema responde con un estado de éxito y redirige de forma automática al usuario hacia la pantalla de inicio de sesión, mostrando una notificación emergente de cuenta creada correctamente.
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el correo electrónico introducido ya se encuentre registrado en el sistema, el sistema deberá interrumpir el	En el caso de que ocurra un fallo de comunicación con la base de datos MySQL al intentar persistir los datos, el sistema del backend capturará la excepción de JPA, el

	proceso de alta, mantener los datos introducidos excepto las contraseñas y mostrar un mensaje de error indicando que el correo ya está en uso.	cliente la recibirá a través de Axios y se mostrará un mensaje de alerta genérico pidiendo al usuario que lo intente de nuevo en unos minutos.
Rendimiento	El sistema deberá procesar la validación, el hash de la contraseña y la inserción en la base de datos en un tiempo máximo de 2 segundos.	
Frecuencia	Este caso de uso se ejecutará de manera esporádica, con una estimación media de 10 a 20 veces al día en fases iniciales de lanzamiento de la aplicación.	
Importancia	Vital, ya que es la puerta de entrada para los nuevos clientes de la plataforma y el paso previo obligatorio para poder realizar pedidos o reservas.	
Urgencia	Inmediatamente, ya que se requiere tener operativas las rutas públicas de captación de usuarios desde el primer despliegue funcional en localhost.	
Comentarios	Durante este flujo el usuario común recibe automáticamente el rol de cliente estándar. Si un usuario desea dar de alta un restaurante con perfil de dueño, el proceso requerirá un formulario extendido o una validación de negocio adicional en el panel de administración.	

3. Buscar restaurantes

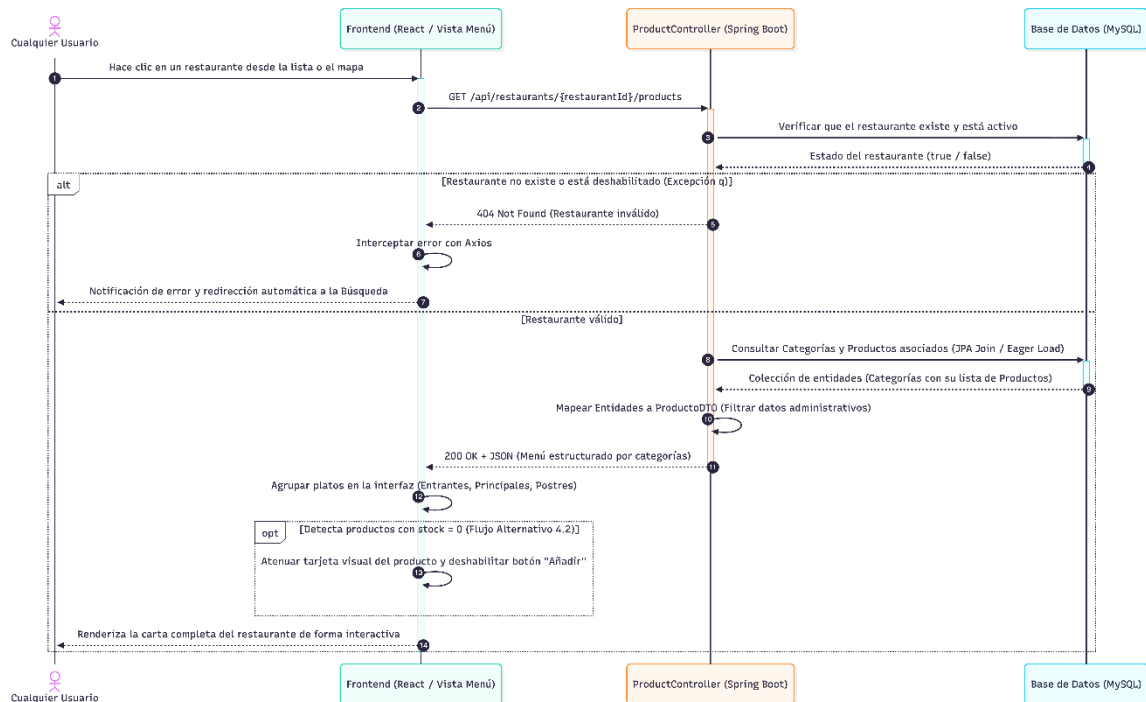


CU-03	Buscar restaurantes en la plataforma
Descripción	Un usuario introduce criterios de búsqueda o navega por el mapa para localizar los restaurantes disponibles. El sistema filtra la base de datos y devuelve un listado con los establecimientos que coinciden con los parámetros especificados.

Actores	Usuario No registrado, Estándar, Admin		
Pre condiciones	El sistema debe tener cargados los datos geográficos y comerciales de los restaurantes en la base de datos MySQL. El frontend debe tener conexión con el servicio de mapas.		
Post condiciones	El usuario visualiza los restaurantes filtrados en la pantalla, pudiendo ver su información básica y seleccionar uno para ver sus productos.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario accede a la sección de búsqueda de restaurantes introduciendo un texto en la barra de búsqueda (como nombre o ciudad) o aplicando filtros por tipo de comida.	El sistema recibe los parámetros de búsqueda a través del cliente web de React.
		1.1. Si el usuario activa la opción de geolocalización por GPS, el sistema obtendrá las coordenadas actuales del navegador del cliente y enviará una petición al backend para listar automáticamente los restaurantes más cercanos en un radio kilométrico definido.	
	2		El backend procesa los criterios de filtrado y realiza una consulta espacial mediante Hibernate Spatial en la base de datos MySQL para encontrar los restaurantes coincidentes.
	3		El sistema devuelve la lista de restaurantes en formato JSON, incluyendo nombres, valoraciones, tipos de cocina y coordenadas geográficas.
	4	El frontend recibe la información y la dibuja dinámicamente en la pantalla utilizando el componente de mapa y una lista de tarjetas visuales.	El sistema muestra los marcadores en el mapa en sus posiciones reales y actualiza el listado lateral con los restaurantes encontrados.
		4.1. Si el usuario limpia todos los filtros o la barra de búsqueda, el sistema recargará la vista mostrando de nuevo todos los restaurantes disponibles de forma general.	
	5	El usuario hace clic sobre un restaurante específico del listado o del mapa.	El sistema procesa la selección y redirige al usuario hacia la vista detallada del menú y productos de ese restaurante.
Excepciones	#	Acción (actor)	Reacción (sistema)

	p	En el caso de que no existan restaurantes en la base de datos que coincidan con los criterios introducidos, el sistema deberá limpiar el mapa de marcadores y mostrar un texto informativo en pantalla que indique: "No se encontraron restaurantes que coincidan con tu búsqueda".	En el caso de que falle el servicio de mapas externo en el navegador, el sistema de React capturará el error de carga de la librería de mapas, pero mantendrá el listado de tarjetas de texto activo para que el usuario pueda seguir interactuando con los restaurantes.
Rendimiento		El sistema deberá realizar la consulta a la base de datos y la devolución del JSON en un tiempo máximo de 1 segundo para garantizar una experiencia de filtrado fluida en tiempo real.	
Frecuencia		Este caso de uso se ejecutará de forma masiva y constante, estimando una media de 200 a 500 búsquedas o actualizaciones de filtros al día.	
Importancia		Vital, ya que es el punto de partida principal del flujo de negocio para que los clientes encuentren dónde consumir o pedir comida.	
Urgencia		Inmediatamente, dado que requiere el correcto acoplamiento de las consultas de localización entre la base de datos y el mapa del frontend.	
Comentarios		Para que las búsquedas por cercanía sean eficientes, se hace uso de las funciones espaciales nativas de MySQL indexadas en la columna de coordenadas, evitando sobrecargar de lógica el servidor de Spring Boot.	

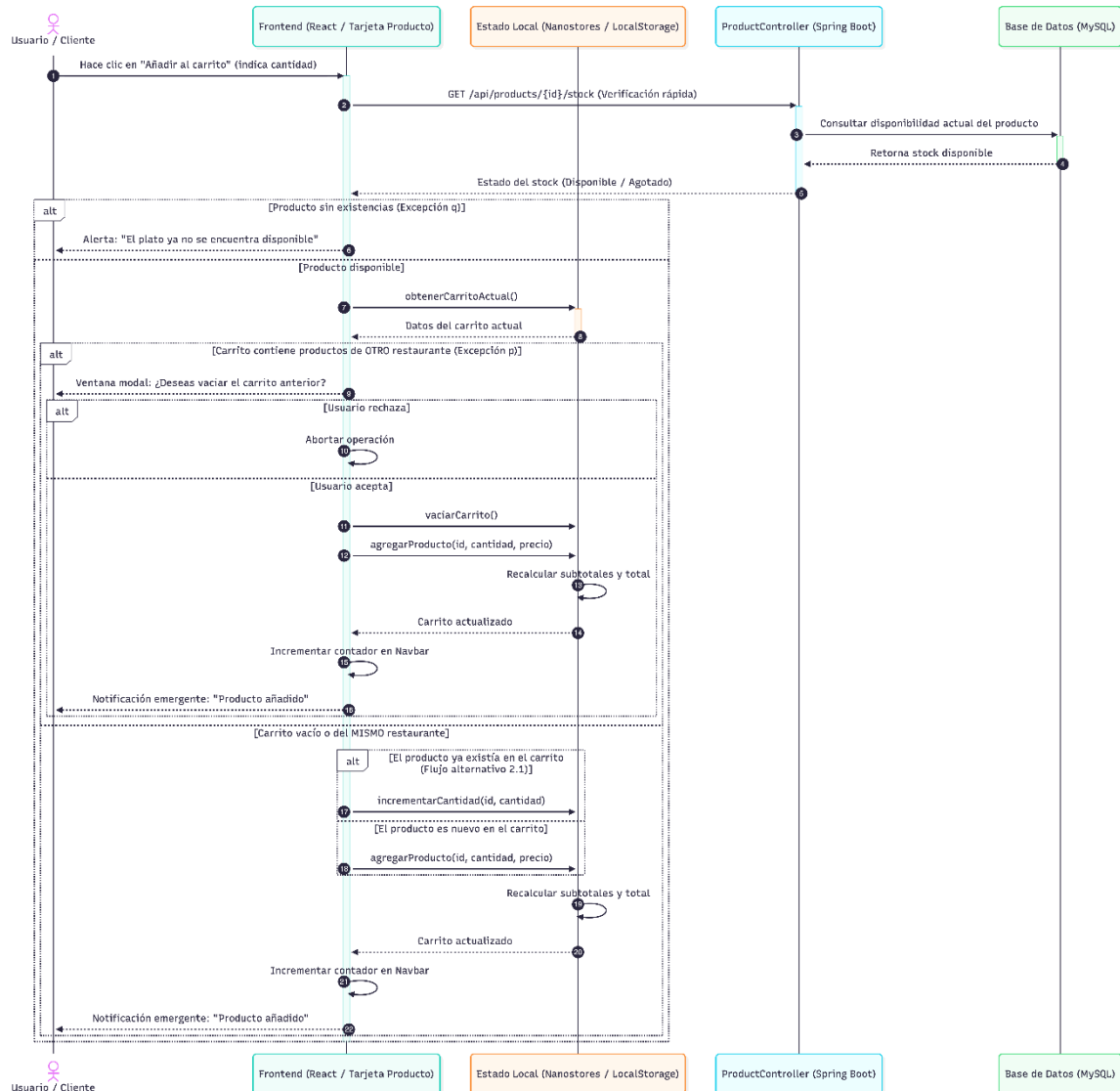
4. Buscar restaurantes



CU-04	Ver productos de un restaurante		
Descripción	El usuario visualiza la lista detallada de productos, platos, bebidas y menús que ofrece un restaurante específico, organizados por categorías. El sistema consulta el catálogo del establecimiento en la base de datos y lo presenta en la interfaz junto con precios, descripciones e ingredientes.		
Actores	Usuario No registrado, Estándar, Admin		
Pre condiciones	El usuario debe haber seleccionado previamente un restaurante activo desde el listado o el mapa de la aplicación.		
Post condiciones	El usuario visualiza el menú del restaurante en la pantalla y queda habilitado para seleccionar productos con el fin de añadirlos a su carrito de compra o interactuar con el pedido.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario hace clic en el restaurante elegido desde la vista de búsqueda o mediante un enlace directo.	El frontend de React detecta el identificador único del restaurante y envía una petición HTTP GET al backend solicitando su catálogo de productos.
	2		El servidor de Spring Boot procesa la solicitud, verifica el estado del restaurante y realiza una consulta a la base de datos MySQL para recuperar los productos asociados y sus categorías correspondientes.
	3		El sistema devuelve la información del menú estructurada en un JSON, incluyendo nombres de platos, precios, descripciones, imágenes y disponibilidad.
	4	El frontend recibe los datos del menú y los renderiza en la pantalla de forma organizada (por ejemplo: Entrantes, Platos Principales, Postres, Bebidas).	El sistema muestra la interfaz gráfica con el catálogo actualizado, destacando los precios y los botones para interactuar con cada artículo.
		4.1. Si el restaurante tiene categorías o secciones especiales definidas por el dueño, el sistema generará un menú de navegación lateral o superior dinámico para que el usuario pueda saltar directamente a la sección que le interese (por ejemplo, comida vegana o celíaca).	4.2. Si un producto específico está marcado en la base de datos como fuera de stock, el sistema mostrará el producto atenuado en la interfaz y deshabilitará el botón de compra para ese artículo en concreto.
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el restaurante seleccionado no tenga ningún	En el caso de que el usuario intente acceder al identificador de un restaurante

	producto registrado en su base de datos, el sistema del frontend mostrará un panel limpio con el mensaje: "Este restaurante aún no ha subido productos a su menú".	que ha sido eliminado o no existe, el backend de Spring Boot devolverá un código de error de recurso no encontrado y el frontend redirigirá al usuario a la pantalla de búsqueda con una notificación de aviso.
Rendimiento	El listado completo de productos de un restaurante debe cargar y renderizarse en la pantalla en un tiempo máximo de 1.2 segundos para evitar la fuga de usuarios.	
Frecuencia	Este caso de uso se ejecutará de forma muy repetitiva, estimando una media de 300 a 600 consultas al día por parte de los clientes de la plataforma.	
Importancia	Vital, ya que es el paso intermedio obligatorio para que la aplicación cumpla su función comercial de venta y gestión de comandas.	
Urgencia	Inmediatamente, dado que requiere coordinar de forma impecable las relaciones de las tablas del modelo de datos de Spring Data JPA (Restaurante - Categoría - Producto).	
Comentarios	Para optimizar la velocidad de este caso de uso y no saturar la base de datos con peticiones pesadas, la consulta del catálogo excluye datos administrativos del restaurante que no sean necesarios para el cliente final	

5. Añadir al carrito



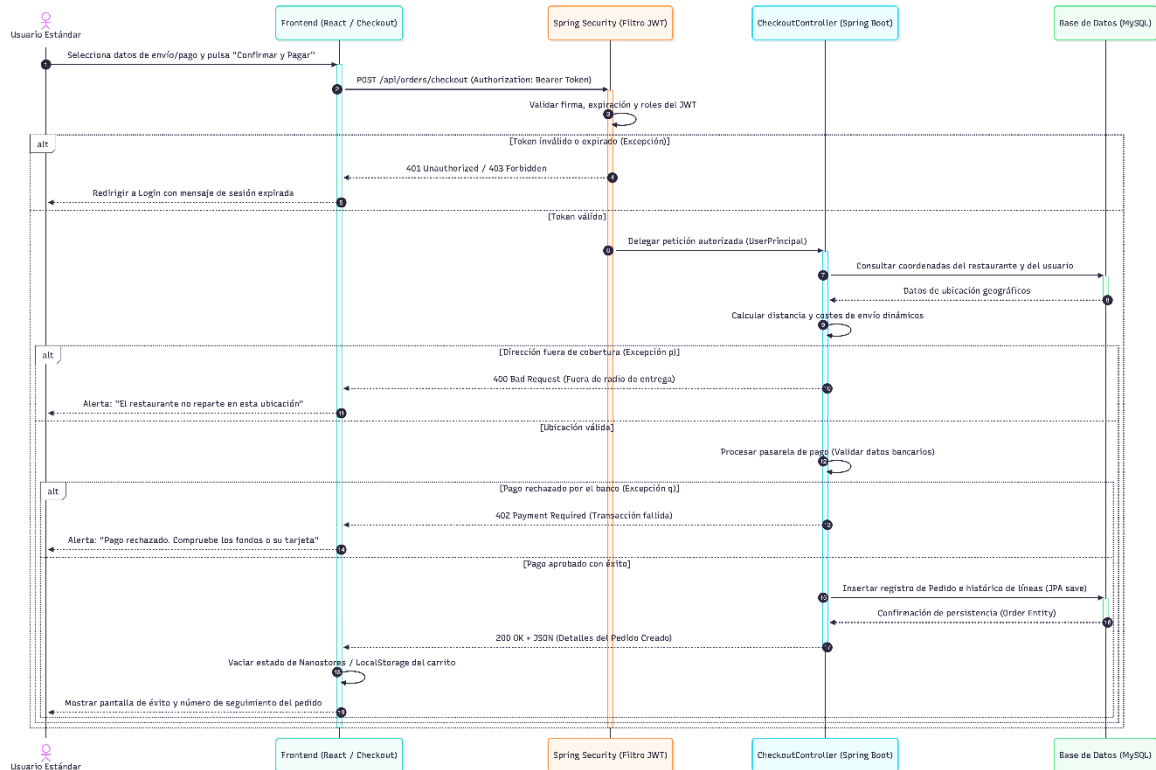
CU-05	Añadir producto al carrito de compra	
Descripción	El usuario selecciona un producto del catálogo de un restaurante, indica la cantidad deseada y lo incorpora a su carrito de compra actual. El sistema calcula los subtotales y actualiza el estado del carrito de forma persistente para la sesión del usuario.	
Actores	Usuario No registrado, Estándar, Admin	
Pre condiciones	El usuario debe estar visualizando el catálogo de productos de un restaurante específico y el producto seleccionado debe estar disponible.	
Post condiciones	El producto y la cantidad seleccionada se guardan en el carrito del usuario. El frontend actualiza los contadores globales y los importes acumulados.	
	#	Acción (actor) Reacción (sistema)

Secuencia Normal	1	El usuario hace clic en el botón de añadir al carrito en la tarjeta de un producto específico.	El frontend de React abre un selector o añade por defecto una unidad del producto, capturando el identificador del artículo y su precio.
	2	El usuario confirma la cantidad y hace clic en aceptar.	El sistema comprueba si ya existe un carrito activo y si el producto pertenece al mismo restaurante que los elementos que ya estuvieran dentro.
		2.1. Si el producto seleccionado ya existía previamente en el carrito, el sistema no duplicará la línea del artículo, sino que sumará la nueva cantidad a la ya existente y recalculará el subtotal de ese producto.	2.2. Si el usuario añade el producto pero aún no ha iniciado sesión, el sistema guardará el carrito de forma temporal en el almacenamiento local del navegador para mantener los productos guardados cuando complete el inicio de sesión
	3		El sistema añade el producto al carrito del usuario (manejado de forma persistente en el estado del cliente con nanostores o sincronizado con el backend) y recalcula el precio total de la orden.
	4		El sistema actualiza la interfaz de la aplicación, incrementando el contador del icono del carrito en la barra de navegación y mostrando una notificación visual de éxito.
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el usuario intente añadir un producto de un restaurante diferente al del carrito actual, el sistema interrumpirá la acción y mostrará un cuadro de diálogo advirtiéndolo: "Ya tienes productos de otro restaurante en el carrito. ¿Deseas vaciar el carrito actual para añadir este plato?". Si el usuario acepta, el sistema limpia el carrito viejo e inicia el nuevo.	En el caso de que el producto se agote en el inventario de la base de datos justo antes de que el usuario haga clic, el sistema del backend rechazará la operación y el frontend mostrará una alerta indicando que el plato ya no se encuentra disponible.
Rendimiento	La actualización del carrito en la interfaz del cliente y el recálculo de los precios globales deben realizarse de forma inmediata en menos de 200 milisegundos.		
Frecuencia	Este caso de uso se ejecutará de manera muy masiva, con una estimación media de 800 a 1500 interacciones diarias en la plataforma.		
Importancia	Vital, representa el paso operativo crucial para acumular los platos elegidos antes de iniciar el proceso de checkout y facturación.		
Urgencia	Inmediatamente, ya que la fluidez de este proceso afecta de forma directa a la conversión de ventas de los restaurantes de la aplicación.		

Comentarios

Para evitar peticiones innecesarias a la base de datos en cada clic de añadir o restar unidades, el estado detallado del carrito se gestiona de forma óptima en memoria del cliente mediante nanostores persistentes, enviándose el bloque estructurado definitivo al servidor únicamente al iniciar el checkout.

6. Checkout

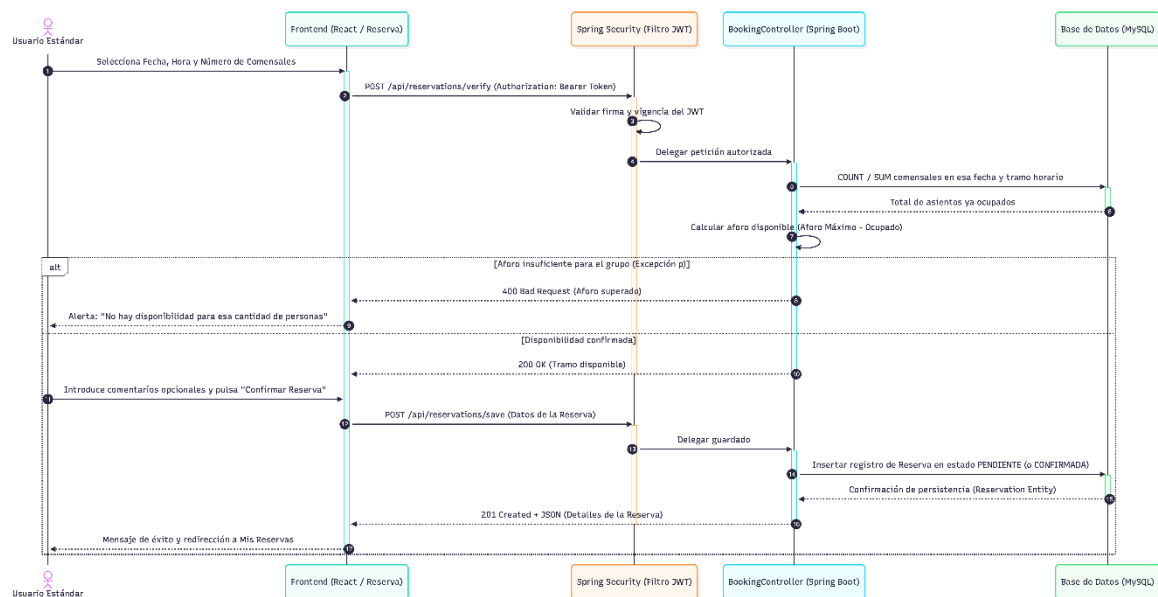


CU-06	Procesar checkout de la compra	
Descripción	El usuario inicia la fase final de la compra para consolidar los productos de su carrito. El sistema unifica los artículos, solicita la información obligatoria de envío y pago, valida la disponibilidad definitiva de la orden y calcula los costes finales de servicio y reparto antes de la creación del pedido.	
Actores	Usuario Estándar, Admin	
Pre condiciones	El usuario debe estar autenticado en la plataforma y debe tener al menos un producto añadido en su carrito de compra.	
Post condiciones	El sistema procesa la información de entrega y de pago de forma válida, dejando la orden lista en el servidor para el cobro y la posterior notificación a las cocinas del restaurante.	
	#	Acción (actor)
		Reacción (sistema)

Secuencia Normal	1	El usuario hace clic en el botón de proceder al checkout desde la vista de su carrito de compra.	El frontend de React empaqueta los datos locales del carrito y realiza una petición HTTP POST hacia el backend de Spring Boot para iniciar el proceso de checkout.
		1.1. Si el usuario intenta iniciar el checkout sin haber iniciado sesión previamente, el sistema interrumpirá el flujo, retendrá los datos del carrito en el almacenamiento local y redirigirá al usuario al caso de uso de iniciar sesión. Una vez autenticado, lo devolverá automáticamente a la pantalla de checkout.	
	2		El sistema bloquea temporalmente los precios del catálogo para la sesión, verifica que el restaurante sigue abierto y solicita al usuario los datos de entrega y el método de pago mediante un formulario resumen.
	3	El usuario selecciona una de sus direcciones guardadas (o introduce una nueva) y escoge su método de pago preferido, confirmando los datos.	El sistema incluye de forma obligatoria las validaciones de la dirección de entrega, calcula la distancia física con el restaurante para aplicar los costes de reparto dinámicos y actualiza el importe total en pantalla.
	4	El usuario revisa el desglose final (productos, tasas de servicio, coste de envío y total) y hace clic en el botón de confirmar y pagar.	El sistema ejecuta el proceso de validación final de la pasarela de pago, limpia el carrito de la sesión local del usuario y delega el flujo en la creación física del pedido en la base de datos MySQL.
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que la dirección de entrega seleccionada por el usuario esté fuera del radio de cobertura o reparto establecido por el restaurante, el sistema mostrará una alerta indicando: "El restaurante elegido no realiza repartos en esta ubicación". El botón de pago final quedará deshabilitado.	En el caso de que la pasarela de pago rechace la transacción (fondos insuficientes, tarjeta caducada o error de autenticación bancaria), el sistema del backend abortará la operación de compra, mantendrá los artículos intactos en el carrito del usuario y mostrará una notificación de error advirtiendo: "El pago no pudo ser procesado. Compruebe los datos de su tarjeta e inténtelo de nuevo".
Rendimiento	El cálculo de costes de envío, impuestos y la carga de la pasarela de pago en el frontend no debe demorarse más de 1.5 segundos en completarse.		

Frecuencia	Se estima que este caso de uso se ejecute una media de 50 a 150 veces al día, dependiendo directamente de la conversión final de los carritos de la aplicación.
Importancia	Vital. Es el núcleo financiero del sistema de pedidos de la plataforma; cualquier fallo aquí detiene por completo los ingresos del negocio.
Urgencia	Inmediatamente, ya que requiere coordinar de forma muy estricta la seguridad de las rutas en Spring Security, el tratamiento de datos sensibles de facturación y las peticiones síncronas.
Comentarios	Este caso de uso engloba obligatoriamente la selección de datos de envío y pago para cumplir con la integridad de los datos requeridos por el modelo del backend antes de guardar el registro definitivo del pedido en las tablas correspondientes.

7. Crear reserva

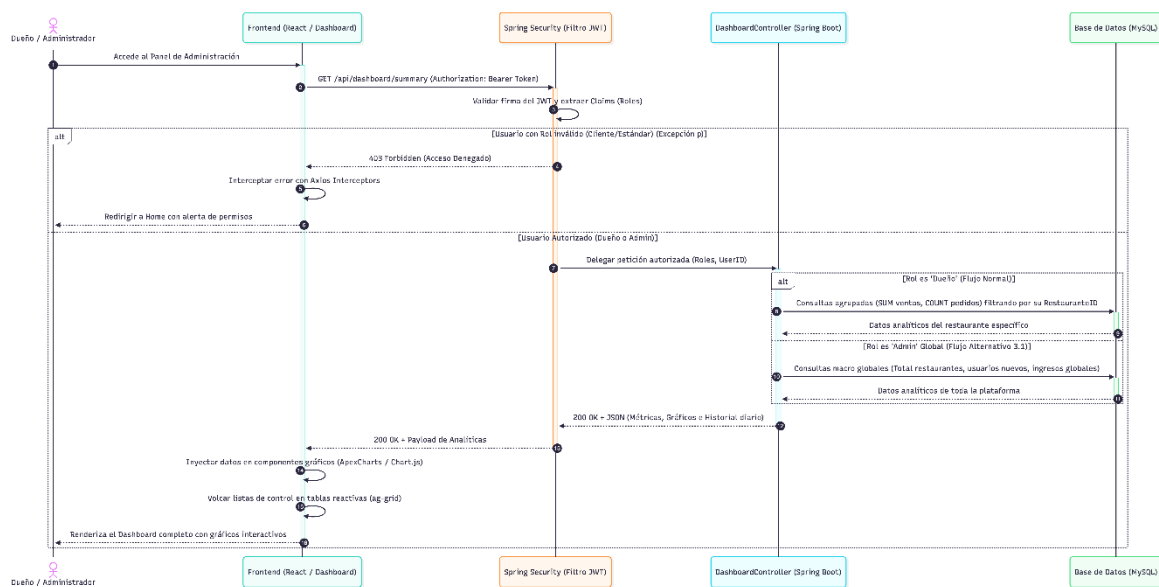


CU-07	Crear reserva de mesa en un restaurante
Descripción	El usuario solicita reservar una mesa en un restaurante para un día, hora y número de comensales específicos. El sistema comprueba el aforo y la disponibilidad en las tablas de la base de datos y, si hay espacio suficiente, registra la reserva en estado pendiente hasta la confirmación de la cocina.
Actores	Usuario Estándar, Admin
Pre condiciones	El usuario debe estar autenticado en la plataforma y debe haber seleccionado un restaurante que disponga del servicio de reservas activo.
Post condiciones	Se registra la reserva con un identificador único en la base de datos MySQL vinculada al usuario y al restaurante, quedando el aforo bloqueado para esa franja horaria.

Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario selecciona la opción de reservar mesa dentro del perfil del restaurante elegido.	El frontend de React despliega un formulario interactivo con un calendario de fechas, un selector de tramos horarios y un contador de comensales.
	2	El usuario introduce la fecha, la hora, el número de personas y hace clic en verificar disponibilidad.	El backend de Spring Boot recibe la petición mediante un HTTP POST y consulta las reservas existentes en MySQL para calcular el aforo ocupado en ese tramo horario específico.
	3		El sistema comprueba que el número de comensales no supera el límite de aforo permitido por el restaurante para esa hora y devuelve una respuesta positiva.
		3.1. Si el restaurante tiene el sistema de confirmación automática activado en su configuración, en el paso 4 el sistema guardará la reserva directamente con el estado de confirmada, en lugar de pendiente.	
	4	El usuario introduce comentarios opcionales (alergias, trona para niños, preferencia de zona) y confirma la creación de la reserva.	El sistema genera el registro de la reserva con el estado de pendiente, resta las plazas del aforo disponible en esa franja horaria y envía una señal de notificación al panel del dueño del restaurante.
		4.1. Si el usuario decide cancelar el proceso antes de confirmar de forma definitiva, el sistema destruirá los datos temporales del formulario y devolverá al usuario a la vista de productos del restaurante sin alterar el aforo.	
Excepciones	5		El frontend actualiza la interfaz mostrando un mensaje de confirmación y redirige al usuario a su historial personal de reservas.
	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el número de comensales solicitado supere el aforo disponible para la fecha y hora elegidas, el sistema bloqueará el botón de confirmación y mostrará un aviso en rojo indicando: "No hay disponibilidad para el número de personas seleccionado en esa franja	En el caso de que el usuario intente seleccionar un día festivo o una hora fuera del horario de apertura comercial definido por el restaurante en su configuración, el sistema deshabilitará esos tramos en el calendario del frontend e impedirá el envío del formulario.

	horaria. Por favor, elija otra hora o reduzca los comensales".	
Rendimiento	El cálculo de aforo disponible y la validación temporal en la base de datos de Spring Boot debe completarse en un tiempo máximo de 1 segundo.	
Frecuencia	Se estima que este caso de uso se ejecute una media de 30 a 80 veces al día, con picos de actividad muy marcados durante los fines de semana.	
Importancia	Importante. Aunque no genera una transacción económica inmediata como el carrito de compra, es una funcionalidad crítica para la fidelización de usuarios y el control de ocupación de los locales.	
Urgencia	Puede esperar a que el núcleo de gestión de pedidos (compras y carrito) esté completamente testeado en localhost.	
Comentarios	Las reservas almacenadas incluyen un sello de tiempo estricto (Timestamp). Si el dueño del restaurante no acepta o rechaza una reserva pendiente antes de que falten 2 horas para la cita, el sistema cuenta con una tarea programada en el backend para marcarla automáticamente como expirada.	

8. Ver dashboard

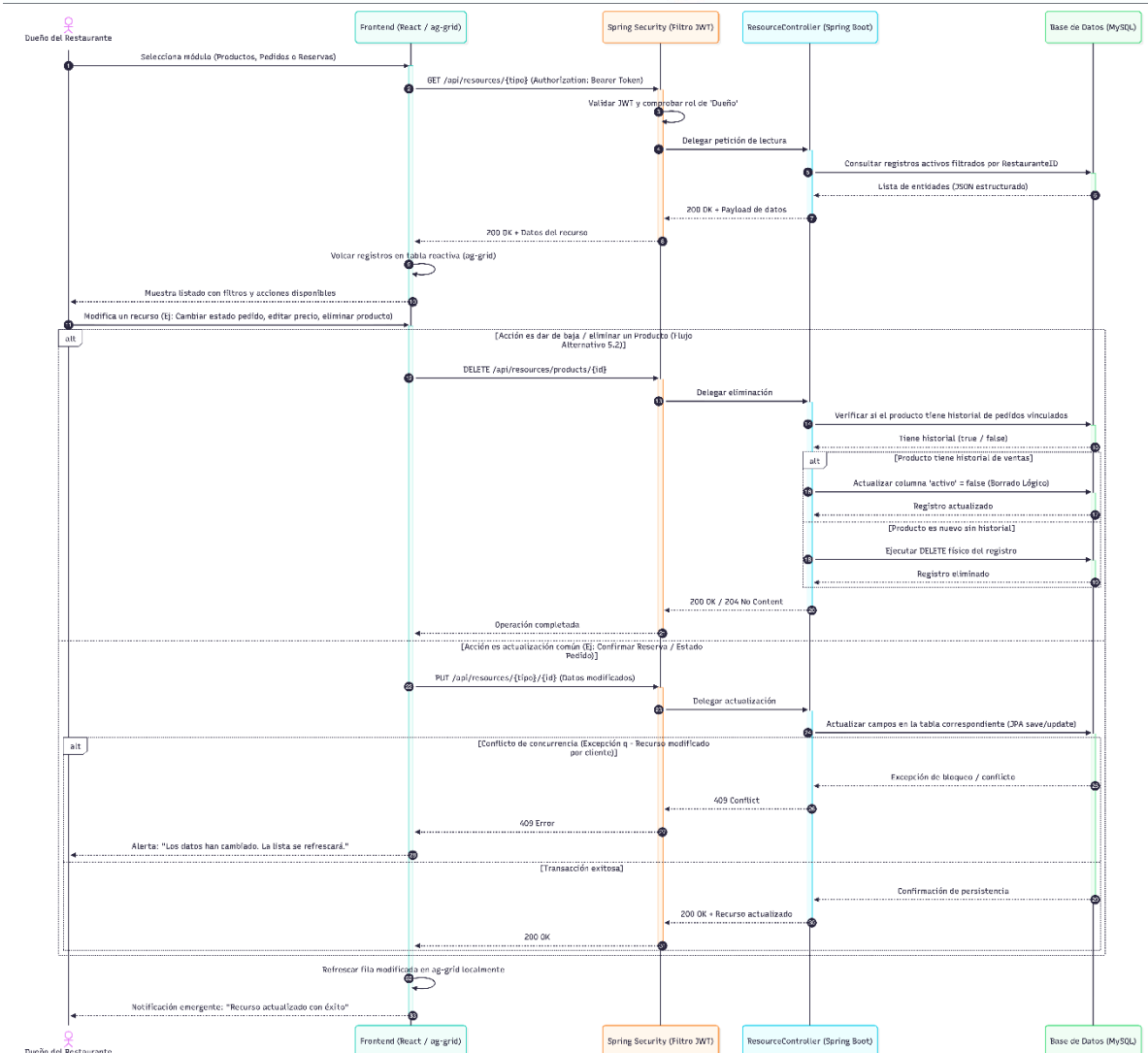


CU-08	Visualizar el panel de control administrativo
Descripción	El usuario accede a la pantalla principal de administración donde el sistema recopila, procesa y muestra de forma agregada métricas clave del negocio, gráficos de rendimiento, resúmenes de reservas y el estado en tiempo real de los pedidos pendientes de gestionar.
Actores	Dueño, Admin

Pre condiciones	El usuario debe estar autenticado en la plataforma y poseer obligatoriamente un rol con privilegios de gestión (Dueño o Administrador) validado por Spring Security.		
Post condiciones	El usuario visualiza la interfaz del dashboard con los datos de analítica de su negocio cargados y actualizados, quedando habilitado para navegar a los módulos específicos de edición y gestión de comandas.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario hace clic en el enlace de acceso al panel de administración en la barra de navegación superior.	El frontend de React intercepta la ruta, lee los roles contenidos en el token JWT y realiza una petición HTTP GET al backend solicitando el resumen de datos del dashboard.
	2	[siguiente paso que hace actor]	El servidor de Spring Boot intercepta la petición a través de sus filtros de seguridad y valida que el usuario tiene los permisos requeridos (RBAC).
	3		El sistema realiza consultas analíticas agrupadas (ventas totales, número de pedidos, reservas activas) en la base de datos MySQL filtrando por el identificador del restaurante del dueño.
		3.1. Si el actor que accede es el 'Admin' (Administrador global de la plataforma), el sistema anulará el filtro por restaurante único y realizará una consulta general para mostrar las métricas globales de toda la aplicación (total de restaurantes dados de alta, ingresos por comisiones de la plataforma y altas de nuevos usuarios).	
	4		El backend unifica las métricas en un objeto JSON optimizado y lo envía de vuelta al cliente con un código de estado de éxito.
	5	El frontend recibe el JSON y distribuye la información a través de componentes gráficos y tablas de datos (como ag-grid).	El sistema renderiza el panel de control mostrando widgets con las métricas del día, gráficos de ingresos y alertas visuales si hay pedidos pendientes en la cocina.
		5.1. Si no hay pedidos ni reservas registradas en el rango de fechas seleccionado, el sistema ocultará las gráficas vacías y mostrará componentes con mensajes ilustrativos indicando que aún no se han registrado operaciones en el periodo de tiempo consultado.	

Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que un usuario con rol de cliente común (Estándar) o un usuario No registrado intente forzar la URL del panel de administración en el navegador, Spring Security denegará la petición del backend devolviendo un código de error de acceso prohibido. El frontend capturará este código y redirigirá de inmediato al intruso a la pantalla de inicio con una alerta de denegación de permisos.	En el caso de que los servicios de cálculo analítico de la base de datos MySQL demoren demasiado en responder debido a un bloqueo temporal de tablas, el backend de Spring Boot lanzará una excepción por tiempo de espera agotado, la aplicación de React la capturará mediante Axios y mostrará un mensaje de alerta solicitando al usuario que actualice el panel.
Rendimiento		Al tratarse de consultas complejas de agregación de datos (operaciones COUNT y SUM), el backend debe resolver las consultas y el frontend pintar las gráficas en un tiempo máximo combinado de 2 segundos.	
Frecuencia		Este caso de uso se ejecutará de forma moderada pero prolongada, estimando una media de 10 a 30 cargas por día por cada dueño de negocio, permaneciendo la vista abierta de fondo para monitorear pedidos entrantes.	
Importancia		Importante. Funciona como la puerta de enlace centralizada y obligatoria desde la cual los gestores se desvían de forma opcional a los casos de uso específicos de modificar horarios, gestionar productos o controlar reservas.	
Urgencia		Hay presión, ya que de la fluidez de este panel depende que los dueños de los restaurantes puedan reaccionar a tiempo a los pedidos y reservas que entran desde la app de los clientes.	
Comentarios		Para asegurar la escalabilidad del sistema y que las consultas analíticas del dashboard no degraden el rendimiento de la base de datos relacional, se crearán índices específicos en MySQL sobre los campos de fecha de creación del pedido y el identificador único del restaurante.	

9. Gestionar recursos (CRUD)

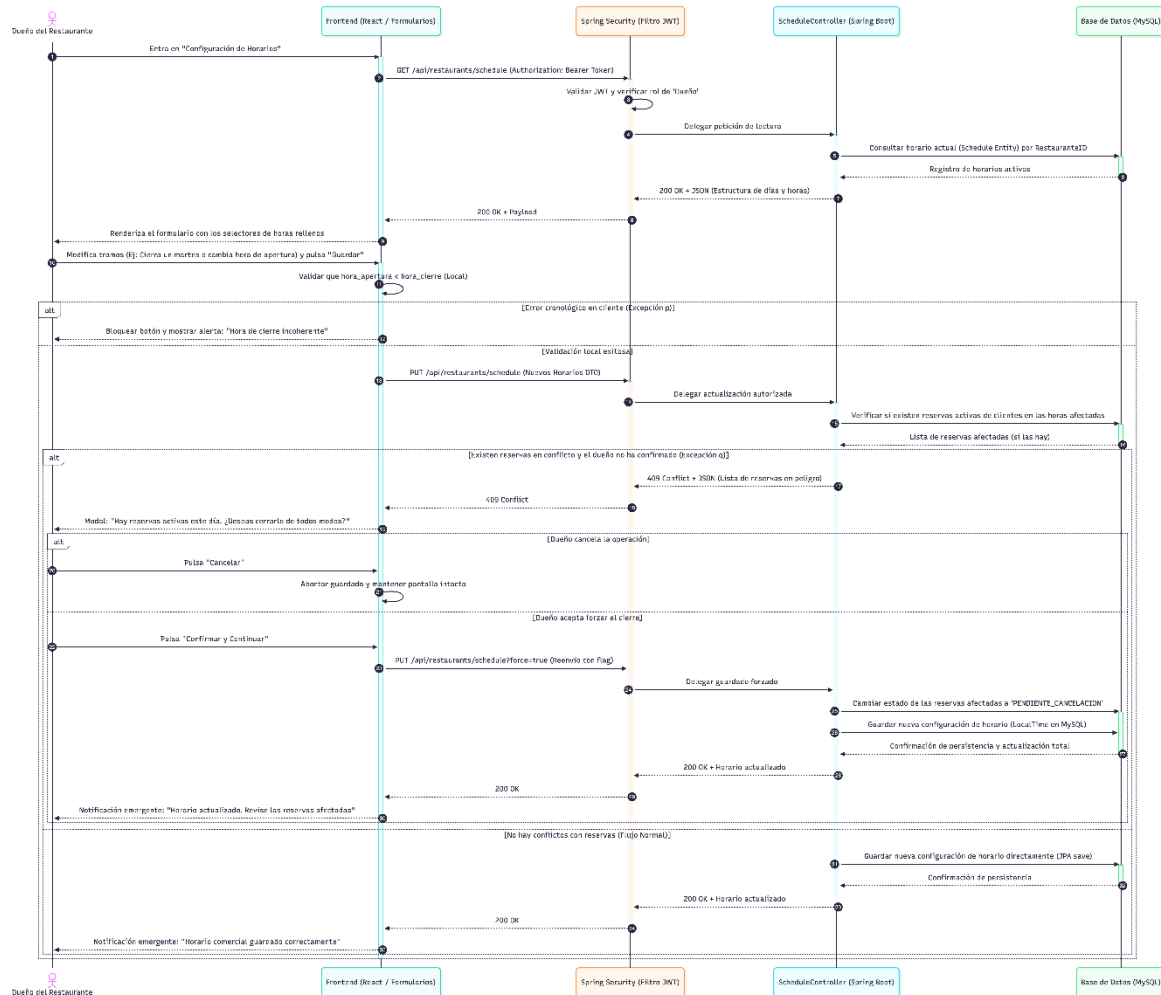


CU-09	Gestionar recursos del restaurante (Reservas, Pedidos y Productos)
Descripción	El usuario con privilegios administra de forma centralizada las operaciones del día a día del restaurante. Esto incluye actualizar los estados de los pedidos entrantes (cocinando, listo, entregado), confirmar o rechazar solicitudes de reserva de mesas, y realizar el mantenimiento CRUD (crear, leer, actualizar, borrar) sobre el catálogo de productos y platos disponibles.
Actores	Dueño, Admin
Pre condiciones	El usuario debe estar autenticado en la plataforma con un rol autorizado, haber accedido previamente a la interfaz de administración y tener un restaurante vinculado a su perfil.
Post condiciones	Los cambios realizados sobre el estado de los pedidos, la ocupación de las mesas o la información del catálogo de productos se actualizan y persisten de forma inmediata en la base de datos MySQL, impactando en tiempo real en la vista pública de los clientes.

Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario selecciona el recurso específico que desea administrar (Reservas, Pedidos o Productos) dentro del menú lateral del panel de control.	El frontend de React realiza una petición HTTP GET dirigida al controlador correspondiente en el backend de Spring Boot para obtener la información actualizada.
	2		El servidor procesa la petición utilizando Spring Data JPA para recuperar los registros actualizados de la base de datos MySQL filtrados por el identificador del restaurante.
	3		El sistema devuelve la lista de datos estructurada en formato JSON hacia el cliente.
	4	El frontend recibe el JSON y vuelca la información en tablas de datos dinámicas con filtros y ordenación avanzados.	El sistema renderiza la tabla en la interfaz, mostrando botones de acción específicos según el tipo de recurso seleccionado.
	5	El usuario realiza una acción operativa sobre uno de los elementos de la lista (por ejemplo, cambiar el estado de un pedido a "Listo", aceptar una reserva pendiente, o editar el precio de un producto) y confirma los cambios.	El frontend empaqueta los campos modificados y envía una petición HTTP (PUT, POST o DELETE) con el token JWT en las cabeceras hacia la API del backend.
		5.1. Si la acción consiste en dar de alta un nuevo plato en la sección de Productos, el sistema abrirá un formulario modal solicitando nombre, precio, descripción y categoría. Al hacer clic en guardar, se validará que los campos obligatorios estén completos antes de realizar la inserción física en la base de datos.	5.2. Si la acción consiste en dar de baja o eliminar un producto, el sistema mostrará un cuadro de diálogo de confirmación. Al aceptar, el backend comprobará si el artículo está asociado a algún pedido histórico; si es así, modificará su estado a oculto o deshabilitado en lugar de realizar un borrado físico, garantizando así la integridad de los datos de facturación.
	6		El backend de Spring Boot valida las reglas de negocio (como asegurar que no se solapen reservas de mesas), ejecuta la actualización de los registros en MySQL y confirma la operación devolviendo una respuesta con éxito.
	7		El frontend refresca la información de la tabla en pantalla de forma reactiva y despliega una notificación emergente indicando que el recurso ha sido actualizado con éxito.

Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el dueño intente cambiar el estado de una reserva a confirmada pero el aforo de esa franja horaria ya se encuentre al límite por reservas previas entrantes, el backend de Spring Boot rechazará la actualización lanzando una excepción de negocio. El frontend capturaré el error y mostrará una alerta advirtiéndolo: "No se puede confirmar la reserva: aforo máximo superado para este tramo horario".	En el caso de que el usuario intente modificar un pedido o reserva que ha sido alterado o cancelado simultáneamente por el cliente desde la aplicación pública, el backend detectará un conflicto de concurrencia de datos y rechazará la petición de guardado. El frontend notificará de inmediato al dueño mediante un aviso en pantalla: "Los datos de este recurso han cambiado. La lista se actualizará automáticamente con la información más reciente".
Rendimiento		La carga inicial de los listados de gestión y la confirmación de operaciones individuales de actualización no deben superar un tiempo máximo de respuesta de 1.2 segundos en condiciones estándar.	
Frecuencia		Al concentrar la gestión operativa diaria, este caso de uso integrado registrará un uso muy intenso, con una estimación media de 100 a 300 interacciones por jornada de trabajo en cada restaurante.	
Importancia		Vital. Es el motor operativo que permite al restaurante procesar el trabajo en cocina, gestionar sus mesas físicas y mantener los precios de la carta debidamente actualizados.	
Urgencia		Inmediatamente, ya que requiere validar e interconectar correctamente las tres entidades principales del modelo relacional del backend de Spring Boot de forma controlada.	
Comentarios		Para ofrecer una experiencia de usuario rápida y profesional en las tablas de visualización de este caso de uso unificado, se utiliza el componente especializado ag-grid-react en el frontend, lo que permite al dueño ordenar, agrupar y filtrar cientos de registros del día (como el histórico de pedidos) localmente en memoria sin castigar al servidor con llamadas repetitivas.	

10. Modificar horario



CU-10	Modificar horario comercial del restaurante	
Descripción	El usuario con privilegios edita la franja horaria de apertura y cierre de su establecimiento, los días de descanso del personal o los turnos específicos de cocina (comidas y cenas). El sistema valida la coherencia de las horas y actualiza el calendario operativo de forma persistente.	
Actores	Dueño, Admin	
Pre condiciones	El usuario debe estar autenticado con un rol autorizado, haber accedido a la sección de configuración de su restaurante dentro del panel administrativo y disponer de permisos de escritura.	
Post condiciones	Las nuevas franjas horarias se guardan en la base de datos MySQL, afectando inmediatamente a las restricciones de los flujos públicos de creación de reservas y pedidos de los clientes.	
	#	Acción (actor)
		Reacción (sistema)

Secuencia Normal	1	El usuario hace clic en la sección de "Configuración de Horarios" dentro del panel de control de su restaurante.	El frontend de React solicita al backend el horario operativo actual del establecimiento mediante una petición HTTP GET.
	2		El servidor de Spring Boot recupera la configuración horaria activa de la base de datos MySQL y la envía en un formato estructurado JSON hacia el cliente.
	3	El frontend recibe los datos y despliega una interfaz con un selector de días de la semana y selectores de tiempo (horas y minutos) para los turnos de apertura y cierre.	El sistema renderiza el formulario mostrando visualmente las franjas horarias actuales del negocio.
	4	El usuario realiza los cambios correspondientes (por ejemplo, adelantar la apertura de las cenas o marcar un día como festivo/cerrado) y hace clic en el botón de guardar.	El frontend valida localmente que las horas de apertura sean anteriores a las de cierre y que los tramos no se solapen.
		4.1. Si el usuario desea establecer el mismo horario para varios días de la semana de forma simultánea, podrá marcar las casillas de verificación de los días deseados en la interfaz y aplicar una plantilla horaria unificada antes de proceder al guardado.	
	5		El cliente envía los nuevos bloques de horarios mediante una petición HTTP PUT hacia la API de Spring Boot.
	6		El backend procesa los cambios, los persiste en la base de datos MySQL y regenera las franjas de disponibilidad para que el sistema de reservas las tome en cuenta de inmediato.
	7		El sistema devuelve una confirmación de éxito y el frontend muestra un aviso emergente indicando que el horario comercial ha sido actualizado correctamente.
Excepciones	#	Acción (actor)	Reacción (sistema)
	p	En el caso de que el usuario introduzca una hora de cierre que sea cronológicamente anterior a la hora de apertura (por ejemplo, apertura a las 23:00 y cierre a las 20:00), el sistema bloqueará el guardado en la interfaz y	En el caso de que el dueño decida cerrar un día completo en el que ya existan reservas de mesas previamente confirmadas por los clientes, el backend de Spring Boot procesará el cambio de horario pero disparará una excepción de

	mostrará una alerta de error indicando: "La hora de cierre no puede ser inferior a la hora de apertura".	advertencia. El sistema mostrará un cuadro de diálogo al dueño indicando: "Existen reservas activas para el día seleccionado. Al cambiar el horario a 'Cerrado', estas reservas pasarán a estado pendiente de cancelación. ¿Desea continuar?". Si el dueño acepta, se aplica el cambio.
Rendimiento	El procesamiento del cambio de configuración, su validación de consistencia en el backend y el guardado en la base de datos relacional debe completarse en un tiempo máximo de 1.5 segundos.	
Frecuencia	Este caso de uso se ejecutará de forma muy esporádica, estimando una media de 1 o 2 veces al mes por establecimiento, utilizándose principalmente en cambios de temporada o periodos vacacionales.	
Importancia	Importante. Aunque su uso no es diario, la correcta definición de este caso de uso es fundamental para alimentar las validaciones de las reglas de negocio de los pedidos en tiempo real y la agenda de reservas del restaurante.	
Urgencia	Puede esperar a que los módulos de CRUD de productos y persistencia de comandas estén completamente implementados y estabilizados en el entorno local.	
Comentarios	Las modificaciones de horario guardadas se indexan y almacenan con formatos estandarizados de hora (LocalTime) en el backend, permitiendo que las consultas de disponibilidad de los clientes (como la del calendario del frontend de reservas) intercepten y consuman estos límites de forma ágil sin sobrecargar la base de datos de llamadas pesadas.	

El análisis de estos flujos permite prever el comportamiento del sistema ante escenarios tanto óptimos como de error, asegurando una experiencia de usuario fluida y una lógica de programación robusta en los servicios del backend.

6.4.4 Mockups de la interfaz

Con el objetivo de definir la experiencia de usuario (UX) y el diseño visual (UI) antes de proceder a la codificación con React y Tailwind CSS, se han diseñado los mockups de las pantallas clave de la aplicación utilizando Figma. Estos bocetos representan la disposición del panel de administración (Dashboard), las páginas web principales y el diseño responsive adaptado a los dispositivos que utilizará el personal de sala.



Inicio

Gestiona tu negocio

Sobre nosotros

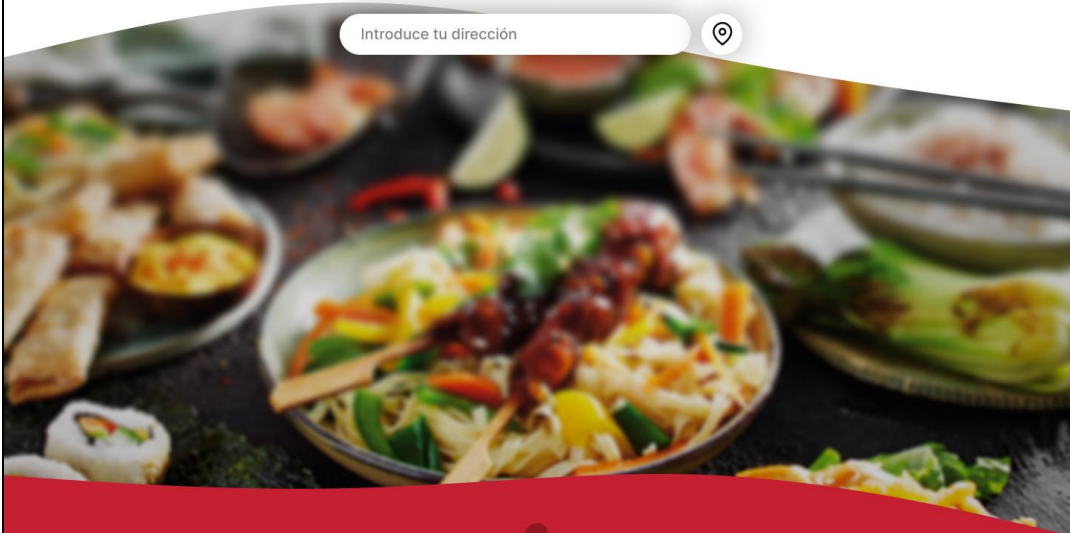
Contacto

Iniciar sesión

Registrarse

¿Donde te llevamos la **comida**?

Introduce tu dirección



Pide también a Domicilio



Glovo



Uber Eats



Just Eat



Deliveroo



To Good To Go



Principales Ciudades en España

Madrid

Barcelona

Valencia

Zaragoza

Sevilla

Murcia

Alicante

Bilbao



Inicio

Pide tu comida

Sobre nosotros

Contacto

Iniciar sesión

Registrarse

La mejor solución para tu negocio de hostelería

Restify es una aplicación que te permite gestionar negocios de hostelería de una manera que nunca antes hubieras imaginado. Es tan simple e intuitivo que no querrás volver atrás. Contamos con las mejores opciones de personalización para adaptarnos de la mejor manera a tu negocio.

Comienza aquí



Glovo

Uber Eats

JUST EAT

deliveroo



stripe



6.1 Presupuesto completo (Hardware, software y recursos humanos)

Para presentar un presupuesto realista, debemos considerar que estamos pasando de un "proyecto de desarrollo" a un "producto mínimo viable (MVP)" listo para operar en un entorno real.

Este presupuesto estima los costes para la **puesta en marcha y el primer año de operación** de un restaurante de tamaño medio o una pequeña cadena.

Concepto	Descripción	Cantidad	Coste Estimado
Terminales Punto de Venta	Tablets de 10" (Android/iPad) para sala (Shadcn UI optimizada).	3	900 €
Pantalla de Cocina (KDS)	Monitor táctil 24" para gestión de comandas en tiempo real.	1	400 €
Impresoras de Tickets	Impresoras térmicas (Bluetooth/Ethernet) para cocina y barra.	2	300 €

Concepto	Descripción	Cantidad	Coste Estimado
Red y Conectividad	Router profesional + Puntos de acceso para evitar microcortes.	1	250 €
Total Hardware			1.850 €

2. Software y Licencias

Costes de las herramientas necesarias para mantener el stack tecnológico activo.

Concepto	Función	Coste Mensual	Coste Anual
Dominio y SSL	Direcciones web (.com, .es) y certificados de seguridad.	-	50 €
Email Corporativo	Google Workspace o Microsoft 365 (Atención al cliente).	10 €	120 €
Herramientas de Diseño	Suscripción a Figma (para prototipado de nuevos widgets).	12 €	144 €
Total Software			314 €

3. Cloud e Infraestructura (Despliegue Docker)

Basado en un despliegue escalable en **AWS (Amazon Web Services)** o **DigitalOcean**.

Servicio	Detalle técnico	Coste Mensual	Coste Anual
Servidor (VPS)	Instancia para correr contenedores Docker (8GB RAM, 4 vCPUs).	40 €	480 €
Base de Datos	MySQL Gestionado (Backups automáticos y alta disponibilidad).	30 €	360 €
Almacenamiento	AWS S3 para fotos de platos y documentos del ERP.	5 €	60 €
CDN (Content Delivery)	Cloudflare para acelerar la carga del Frontend React.	0 € (Free tier)	0 €
Total Cloud			900 €

4. Recursos Humanos (Desarrollo y Mantenimiento)

Este es el coste de "crear" el software antes de la puesta en marcha (estimación de 4 meses de desarrollo MVP).

Perfil	Responsabilidad	Esfuerzo	Coste Total
Fullstack Developer	Desarrollo de Spring Boot + React	4 meses	8.000 €
Diseñador UX/UI	Configuración de Shadcn, Tailwind y flujo de usuario.	2 semanas	850 €
DevOps / SysAdmin	Configuración de Docker, CI/CD y seguridad Cloud.	2 semanas	1.100 €
Total RRHH			9.950 €

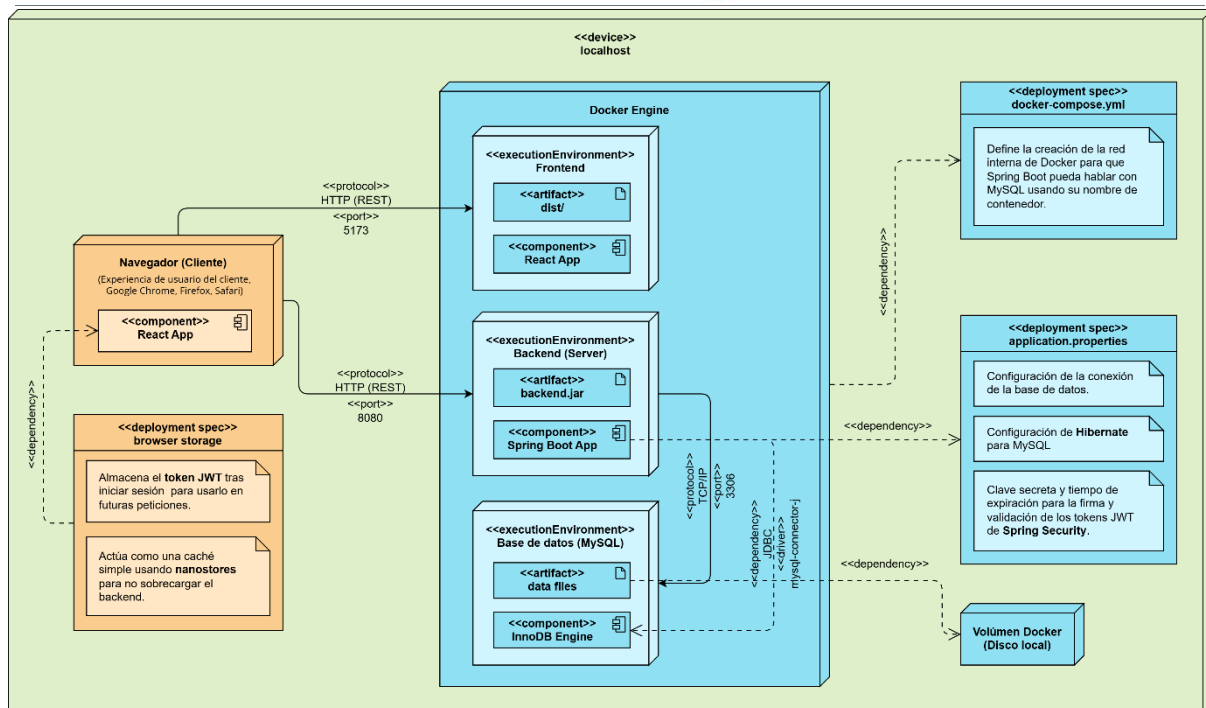
Resumen Final del Presupuesto (Año 1)

- Inversión Inicial (CAPEX):** 11.800 € (Hardware + Desarrollo inicial) **Coste Operativo Anual (OPEX):** 1.214 € (Cloud + Licencias)
- PRESUPUESTO TOTAL (Aproximado): 13.014 €**

7. Desarrollo y ejecución

Para comprender la infraestructura tecnológica y la distribución física de los componentes que dan vida a **Restify**, es necesario analizar su arquitectura de despliegue. Al tratarse de una aplicación web moderna y escalable, se ha optado por un modelo desacoplado basado en contenedores independientes mediante **Docker**. Esto garantiza el aislamiento del entorno, la facilidad de mantenimiento y la paridad entre los entornos de desarrollo y producción.

El siguiente **diagrama de despliegue** detalla cómo interactúa el cliente (navegador web) con los nodos que alojan el servidor de aplicaciones, la lógica de negocio y la persistencia de datos.



7.1 Descripción General del Funcionamiento

El flujo operativo de la aplicación sigue una arquitectura cliente-servidor basada en el protocolo HTTP/S y peticiones REST:

- **Capa de Presentación (Frontend):** El usuario interactúa con una interfaz SPA (*Single Page Application*) desarrollada en **React** y construida con **Vite**, estilizada mediante **Tailwind CSS**. Este componente se sirve al cliente y se ejecuta directamente en su navegador web.
- **Puerta de Enlace y Seguridad (API REST):** Todas las peticiones del frontend se dirigen al backend desarrollado en **Spring Boot**. El primer filtro del sistema es la capa de seguridad (**Spring Security**), que intercepta las solicitudes, valida la presencia de un token **JWT** (JSON Web Token) válido en la cabecera y verifica que el rol del usuario (`user_roles`) tenga permisos para el recurso solicitado (mecanismo RBAC).
- **Capa de Negocio y Persistencia:** Una vez autorizada la petición, los controladores (*Controllers*) derivan la lógica a los servicios correspondientes (gestión de comandas, analíticas del dashboard, disponibilidad horaria, etc.). Los servicios interactúan con la base de datos a través de repositorios de **Spring Data JPA**.
- **Base de Datos:** Los datos se almacenan de forma persistente en un servidor **MySQL**. Como se observa en el diagrama, este nodo está aislado y solo es accesible por el contenedor del backend, garantizando la seguridad de la información.

7.2 Dificultades y resoluciones técnicas

A lo largo del ciclo de vida del desarrollo de **Restify**, surgieron diversos desafíos técnicos derivados de la complejidad de los requisitos, tales como la gestión de sesiones seguras sin estado, la estructura de base de datos para analíticas dinámicas y la configuración de entornos aislados. A continuación, se documentan las soluciones técnicas implementadas para resolver estos puntos críticos.

7.2.1. Inestabilidad visual de fondos vectoriales (SVG) en la Landing Page

- **Problemática:** Durante las pruebas de diseño responsive y accesibilidad en la página de aterrizaje (*landing page*), se detectó que los fondos vectoriales con formas de onda (*wave SVGs*) sufrían deformaciones severas al aplicar zoom en el navegador o al renderizarse en pantallas con relaciones de aspecto ultra-panorámicas. El vector se estiraba de forma desproporcionada en el eje vertical, descolocando los contenedores de texto, superponiéndose a los botones de acción y rompiendo el flujo visual de la interfaz.
- **Solución implementada:** Se rediseñaron los archivos SVG modificando sus propiedades internas de encapsulamiento. Se eliminaron las dimensiones fijas en píxeles y se configuró correctamente el atributo `viewBox` junto con la directiva `preserveAspectRatio="xMidYMid slice"`. En las hojas de estilo de **Tailwind CSS**, se establecieron propiedades de escala basadas en unidades del viewport (`vh` y `vw`), forzando al gráfico vectorial a bloquear su altura (*height*) y reescalar únicamente su anchura de manera elástica. Esto garantizó que el fondo actúe como una capa decorativa fluida y estable sin importar el nivel de zoom del usuario.

7.2.2. Inconsistencia en el consumo de APIs de Geolocalización externa

- **Problemática:** Al intentar implementar la funcionalidad de autocompletado de direcciones y posicionamiento del usuario, las llamadas directas a las APIs de geolocalización basadas en texto libre devolvían resultados ambiguos, incompletos o erróneos, a pesar de estructurar rigurosamente los parámetros sintácticos en las URLs de petición (*query params*). Nombres de calles duplicados en diferentes regiones de España provocaban que el sistema ubicara al usuario en una provincia incorrecta, imposibilitando el cálculo logístico posterior.
- **Solución implementada:** Se rediseñó el subsistema de localización implementando un flujo híbrido de **Geocoding** (conversión de dirección postal a coordenadas) y **Reverse Geocoding** (conversión de coordenadas a dirección normalizada). El sistema pasó a depender principalmente de coordenadas geográficas precisas (latitud y longitud en formato `double`), capturadas mediante la API de geolocalización nativa del navegador del usuario (con su previo consentimiento). Estas coordenadas se envían al backend para realizar una consulta inversa cruzada, asegurando una resolución de datos de dirección 100% fiable y unívoca antes de persistirlos en la tabla `addresses`.

7.2.3. Limitación espacial en la delimitación de zonas de reparto (*Delivery Radius*)

- **Problemática:** Inicialmente, el modelo de negocio contemplaba la asignación de reparto a nivel de municipio o ciudad. Sin embargo, este enfoque resultó inviable para grandes urbes (como Madrid o Sevilla), donde el perímetro municipal se extiende decenas de kilómetros. Permitir un pedido desde un extremo de la ciudad a otro introducía tiempos de tránsito superiores a una hora, degradando la calidad del producto (comida fría) y elevando drásticamente los costes logísticos del restaurante.
- **Solución implementada:** Se sustituyó el criterio geopolítico por un modelo de delimitación espacial radial. Se añadió el atributo `delivery_radius` (en kilómetros) a la entidad `restaurants`. Para validar si un cliente es apto para el reparto a domicilio, el backend en Spring Boot intercepta la coordenada de la dirección del cliente (`addresses`) y la del restaurante (`restaurants`), y calcula la distancia lineal entre ambos puntos sobre la superficie terrestre aplicando la **fórmula de Haversine**:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\varphi}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Donde:

- R es el radio de la Tierra (6371 km),
- φ_1, φ_2 son las latitudes de ambos puntos en radianes.
- $\Delta\varphi$ y $\Delta\lambda$ son las diferencias de latitud y longitud respectivamente.

Si la distancia calculada d es menor o igual al `delivery_radius` del establecimiento, el pedido es aprobado por el sistema; en caso contrario, se notifica al usuario que se encuentra fuera de la zona de cobertura.

2.4. Complejidad en la persistencia de estados dinámicos de la interfaz (React Grid Layout)

- **Problemática:** La personalización del panel analítico de cada restaurante permite a los gerentes arrastrar, soltar y cambiar el tamaño de los componentes visuales a través de la librería **React Grid Layout**. Esta librería genera un árbol de objetos JSON altamente complejo que describe la cuadrícula (coordenadas X e Y, anchura, altura y propiedades de redimensión de cada elemento). Intentar normalizar esta estructura en el modelo relacional para cumplir con las Formas Normales habría requerido la creación de múltiples tablas auxiliares, provocando un impacto negativo en el rendimiento de la base de datos debido a las costosas operaciones de escritura y múltiples *JOINS* cada vez que un usuario modificaba su panel.
- **Solución implementada:** Aprovechando las capacidades de almacenamiento flexible de MySQL, se optó por una estrategia de persistencia híbrida. En lugar de descomponer el objeto en entidades relacionales, se definieron los campos `layouts` en la tabla `widgets` y `tabs` en la tabla `dashboard_pages` utilizando el tipo de datos **JSON/JSONB** nativo del motor de base de datos. De este modo, el frontend en React serializa de forma directa el estado de la cuadrícula en un único string JSON y lo envía al backend, donde Spring Boot lo almacena de manera atómica a través de JPA. Esta solución redujo a cero la redundancia de datos, simplificó la lógica del API REST y optimizó los tiempos de carga del Dashboard.

8. Conclusiones y reflexiones

8.1 Valoración global del proyecto

La culminación de **Restify** representa la materialización de una plataforma integral, robusta y escalable que responde de manera efectiva a los desafíos reales que afronta el sector de la hostelería actual. El desarrollo de este ecosistema *multi-tenant* ha permitido demostrar la viabilidad de unificar la gestión interna de un restaurante (comandas, reservas y disponibilidad horaria) con un panel analítico dinámico y altamente personalizable por el usuario.

Desde una perspectiva técnica, el proyecto ha cumplido con éxito todos los objetivos de diseño y arquitectura planteados inicialmente. La separación estricta de responsabilidades mediante una API REST desarrollada en Spring Boot y una interfaz SPA ágil en React ha demostrado ser una elección arquitectónica idónea, garantizando un bajo acoplamiento y un rendimiento óptimo. Asimismo, la resolución de desafíos críticos —tales como la persistencia de estructuras complejas mediante tipos JSON en MySQL, la seguridad perimetral basada en el Control de Acceso Basado en Roles (RBAC) con JWT, o el control espacial del reparto a través de la fórmula de Haversine— aporta al proyecto un valor técnico diferenciador que trasciende el ámbito académico, acercándolo a un estándar de producto de software comercializable.

En definitiva, **Restify** se consolida como una solución moderna y eficiente que simplifica los flujos de trabajo diarios en la restauración y dota a la gerencia de una herramienta analítica fundamental para la toma de decisiones estratégicas.

8.2 Resumen de lo aprendido

El desarrollo de este proyecto ha supuesto la experiencia de aprendizaje más enriquecedora y exigente de todo el ciclo. Más allá de consolidar los conocimientos teóricos adquiridos en las aulas, el proyecto ha obligado a profundizar de manera autónoma en tecnologías de vanguardia y en la resolución de problemas de ingeniería de software complejos.

Entre los aprendizajes técnicos y metodológicos más significativos, destacan los siguientes:

- **Dominio de Arquitecturas Modernas Desacopladas:** Se ha interiorizado el flujo completo de comunicación asíncrona entre un frontend reactivo y un backend empresarial, comprendiendo la importancia de diseñar APIs RESTful limpias, autodocumentadas y estructuradas en capas (Controladores, Servicios, Repositorios).
- **Gestión Avanzada de Persistencia Relacional e Híbrida:** El diseño de la base de datos ha permitido ir un paso más allá de la normalización tradicional, aprendiendo a evaluar el impacto del rendimiento y a implementar soluciones híbridas (como el almacenamiento de configuraciones de la interfaz en formato JSON directo sobre PostgreSQL) para optimizar las lecturas y escrituras del sistema.
- **Seguridad y Control de Acceso:** La implementación práctica de un flujo de autenticación y autorización mediante JSON Web Tokens (JWT) y Spring Security ha proporcionado una comprensión profunda sobre la protección de recursos críticos, la gestión de sesiones sin estado (*stateless*) y la jerarquía de permisos basada en roles dentro de una organización.
- **Pensamiento Algorítmico Aplicado a la Logística:** Resolver problemas de restricciones espaciales reales mediante la aplicación de fórmulas matemáticas como la de Haversine ha supuesto un excelente ejercicio para entender cómo trasladar reglas de negocio físicas a código de software preciso.
- **Metodología, Despliegue y Autonomía:** El uso de metodologías ágiles para estructurar el desarrollo incremental y la contenerización de los entornos mediante Docker ha revelado la vital importancia de los procesos de DevOps en el ciclo de vida del software. Asimismo, el diagnóstico de errores complejos (como los desajustes de escalado vectorial SVG o las inconsistencias en las APIs de geolocalización) ha fortalecido la capacidad de resiliencia y la aptitud para la investigación técnica independiente.

Como conclusión final, este proyecto no solo ha certificado las competencias técnicas necesarias para el diseño, desarrollo y despliegue de aplicaciones multiplataforma, sino que ha forjado la mentalidad analítica y la capacidad de adaptación requeridas para integrarse con éxito en el sector tecnológico profesional.

8.3 Futuras mejoras y ampliaciones

A pesar de que **Restify** cumple con solvencia todos los objetivos iniciales del MVP y presenta un ecosistema funcional completo, la naturaleza dinámica del sector de la restauración y las demandas de escalabilidad tecnológica abren las puertas a una serie de optimizaciones. A continuación, se detallan las mejoras planificadas para el sistema, clasificadas en infraestructura/seguridad y evolución funcional.

8.3.1. Infraestructura, rendimiento y seguridad

- **Caché distribuida con Redis:** Con el fin de reducir la latencia en las peticiones concurrentes y mitigar la carga sobre la base de datos MySQL, se plantea la integración de **Redis** como capa de caché en memoria. Esto optimizará el almacenamiento de sesiones web, los roles de usuario y las configuraciones estáticas de los layouts de los dashboards.
- **Migración de entornos y despliegue avanzado:** Actualmente el sistema está containerizado de manera local. Se proyecta el despliegue del frontend en plataformas *serverless* de alto rendimiento como **Vercel** o **Netlify**, mientras que el backend se alojará en un Servidor Virtual Privado (**VPS**) optimizado.
- **Desacoplamiento de almacenamiento y Cloud CDN:** Para garantizar una alta disponibilidad de los datos y una entrega de archivos ultra-rápida (como las imágenes de los productos o los avatares de los usuarios), se migrará la base de datos a un servicio gestionado como **Amazon RDS**. Paralelamente, se implementará **Amazon S3** para el almacenamiento de archivos multimedia junto con **Cloudflare** como red de distribución de contenidos (CDN) y escudo de protección contra ataques DDoS.
- **Cifrado extremo a extremo y auditoría avanzada:** Se implementará un sistema robusto de rotación automatizada de secretos (JWT secret keys) y auditoría (*logging*) avanzada a través de herramientas como ELK Stack (Elasticsearch, Logstash y Kibana), asegurando

8.3.2. Evolución funcional del ecosistema

- **Integración con plataformas de Delivery.** Con el objetivo de unificar la gestión de pedidos en una única pantalla, se desarrollarán integraciones con las APIs de servicios externos de reparto como **Glovo** o **Uber Eats**. Esto evitará que los restaurantes tengan que duplicar la gestión de comandas en diferentes dispositivos.
- **Pasarelas de pago reales:** Se sustituirá el flujo simulado de facturación por una integración real con pasarelas de pago digitales (como **Stripe** o **PayPal**), permitiendo cobros seguros mediante tarjetas de crédito, Apple Pay y Google Pay.
- **Módulo de TPV Físico:** Se evolucionará la plataforma para incluir un software de **Terminal Punto de Venta (TPV)** táctil adaptado a pantallas de sala, permitiendo la sincronización inmediata de la barra y la cocina con el inventario y el dashboard analítico.
- **Carta digital mediante códigos QR:** Se añadirá una funcionalidad de generación dinámica de códigos QR para las mesas del restaurante. Los comensales podrán escanear el código para acceder a una carta virtual interactiva, filtrada por alérgenos y con actualización de precios en tiempo real.
- **Auto comandas y pago integrado desde el móvil:** Vinculado al punto anterior, se habilitará la posibilidad de que el cliente realice su propio pedido (*self-ordering*) y efectúe el pago de su cuenta directamente desde su dispositivo móvil sin necesidad de ser atendido presencialmente por un camarero, agilizando la rotación de mesas.
- **Módulo avanzado para flota de repartidores propia:** El sistema actual opera bajo un modelo básico de reparto local. Se diseñará un módulo logístico avanzado con una aplicación móvil nativa para repartidores propios, incluyendo asignación automatizada de rutas por proximidad, seguimiento por GPS en tiempo real y cálculo de tiempos estimados de entrega (ETA).

9. Bibliografía e Webgrafía

- *Spring boot.* (s. f.). Spring Boot. <https://spring.io/projects/spring-boot>

- React. (s. f.). <https://es.react.dev/>
- Vite. (s. f.). Vitejs. <https://vite.dev/>
- Docker: Accelerated Container Application Development. (2026, 27 mayo). Docker. <https://www.docker.com/>
- Spring Security. (s. f.). Spring Security. <https://spring.io/projects/spring-security>
- Spring Data JPA. (s. f.). Spring Data JPA. <https://spring.io/projects/spring-data-jpa>
- Project Lombok. (s. f.). <https://projectlombok.org/>
- Halterman, J. (s. f.-b). ModelMapper - Simple, intelligent, object Mapping. <https://modelmapper.org/>
- Auth. (s. f.). JWT.IO. JSON Web Tokens - jwt.io. <https://www.jwt.io/>
- Mattjohnsonpint. (s. f.). GitHub - mattjohnsonpint/GeoTimeZone: Provides an IANA time zone identifier from latitude and longitude coordinates. GitHub. <https://github.com/mattjohnsonpint/GeoTimeZone>
- axios | Promise based HTTP client. (s. f.). <https://axios.rest/>
- React Router Official Documentation. (s. f.). <https://reactrouter.com/>
- React-Grid-Layout. (s. f.). GitHub - react-grid-layout/react-grid-layout: A draggable and resizable grid layout with responsive breakpoints, for React. GitHub. <https://github.com/react-grid-layout/react-grid-layout>
- Haas, R., & KingSora. (s. f.). OverlayScrollbars. <https://kingsora.github.io/OverlayScrollbars/>
- bvaughn. (s. f.). GitHub - bvaughn/react-resizable-panels. GitHub. <https://github.com/bvaughn/react-resizable-panels>
- Shadcn. (s. f.). The Foundation for your Design System - shadcn/ui. <https://ui.shadcn.com/>
- Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. (s. f.). Tailwind CSS. <https://tailwindcss.com/>
- Lucide icons. (s. f.). Lucide. <https://lucide.dev/>
- Lodash. (s. f.). <https://lodash.com/>

- Maps for developers. (s. f.). MapTiler. <https://www.maptiler.com/>
- Leaflet — an open-source JavaScript library for interactive maps. (s. f.). <https://leafletjs.com/>
- Google Gemini. (s. f.). Gemini. <https://gemini.google.com/app>
- Redis. (s. f.). Redis - Real-time Data For Agents & Apps. <https://redis.io/>
- Stripe. (s. f.). Stripe | Infraestructura financiera para tu negocio. <https://stripe.com/es>
- Enviar dinero | Cartera digital y gestión de dinero. (s. f.). PayPal. <https://www.paypal.com/es/home>
- Vercel: Build and deploy the best web experiences with the AI Cloud. (s. f.). Vercel. <https://vercel.com/>
- Push your ideas to the web | Netlify. (s. f.). Netlify. <https://www.netlify.com/>
- Glovo. (s. f.). Glovoapp. <https://glovoapp.com/>
- Uber Eats | Food & Grocery Delivery | Order Groceries and Food Online. (s. f.). <https://www.ubereats.com/>
- Apple. (s. f.). Apple Pay. Apple (España). <https://www.apple.com/es/apple-pay/>
- Google Pay - Paga online, en tiendas o envía dinero sin complicaciones. (s. f.). Google Pay - Paga online, en tiendas o envía dinero sin complicaciones. Google Pay. https://pay.google.com/intl/es_es/about/
- Amazon S3 - Almacenamiento de objetos en la nube - AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/s3/>
- Base de datos SQL administrada - Amazon Relational Database Service (RDS) - AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/rds/>
- Conecta, protege y desarrolla en cualquier parte. (s. f.). Cloudflare. <https://www.cloudflare.com/es-es/>
- MySQL. (s. f.). MySQL. <https://www.mysql.com/>
- Workspace, G. (s. f.). Business Apps y herramientas de colaboración | Google Workspace. Google Workspace. <https://workspace.google.com/intl/es/>

- IA para aumentar la productividad empresarial | Microsoft 365 Copilot. (s. f.).
<https://www.microsoft.com/es-es/microsoft-365-copilot/enterprise>
- Figma. (s. f.). Figma | La herramienta de diseño de interfaz colaborativa.
<https://www.figma.com/es-es/>
- Android | Haz más con Google en teléfonos y dispositivos Android. (s. f.). Android.
https://www.android.com/intl/es_es/
- Mermaid. (s. f.). Diagramming And Charting Tool. <https://mermaid.js.org/>

10. Anexo I – Manual técnico de instalación o puesta en marcha

(Ver archivo install.docx / install.pdf)

11. Anexo II – Documentación de uso (manuales de usuario)

(Ver archivo user.docx / user.pdf)